

```

1  from __future__ import annotations
2  from pathlib import Path
3  from typing import List, Tuple
4  import math
5  import numpy as np
6  from PIL import Image, ImageFilter
7  from .models import CrackObject, WidthSample
8  from .geometry import polyline_length, distance
9  from .severity import CrackSeverityEvaluator
10 Point = Tuple[float, float]
11 class CrackAutoDetector:
12     """自动裂缝标注算法。
13     本版不再使用“全局暗像素连通”作为主逻辑，而是采用：
14     1. 局部背景减法，增强“比周边更暗”的线状裂缝；
15     2. 邻域连续性过滤，去掉单个石子、纹理噪点；
16     3. 连通域几何筛选，过滤块状污渍、边缘噪声和短小纹理；
17     4. 线性支撑度验证，确认拟合折线确实落在暗线区域上；
18     5. 默认保守识别，避免一次性生成大量错误标注线。
19     """
20     PRESETS = {
21         "保守": {
22             "contrast_percentile": 96.0,
23             "min_contrast": 18,
24             "min_length_px": 130,
25             "min_area": 14,
26             "max_count": 6,
27             "support": 0.42,
28             "edge_guard": 0.035,
29             "neighbor_min": 2,
30             "line_width": 2,
31         },
32         "标准": {
33             "contrast_percentile": 94.0,
34             "min_contrast": 14,
35             "min_length_px": 95,
36             "min_area": 10,
37             "max_count": 8,
38             "support": 0.37,
39             "edge_guard": 0.025,
40             "neighbor_min": 2,
41             "line_width": 2,
42         },
43         "增强": {
44             "contrast_percentile": 91.0,
45             "min_contrast": 11,
46             "min_length_px": 70,
47             "min_area": 8,
48             "max_count": 12,
49             "support": 0.30,
50             "edge_guard": 0.018,

```

```

1         "neighbor_min": 1,
2         "line_width": 3,
3     },
4 }
5 @staticmethod
6 def detect_from_image(
7     image_path: str | Path,
8     meter_per_pixel: float | None = None,
9     max_cracks: int | None = None,
10    min_length_px: float | None = None,
11    sensitivity: str = "保守",
12) -> List[CrackObject]:
13    arr = CrackAutoDetector._load_gray(image_path)
14    if arr is None:
15        return []
16    cfg = dict(CrackAutoDetector.PRESETS.get(sensitivity, CrackAutoD
17    if max_cracks is not None:
18        cfg["max_count"] = max_cracks
19    if min_length_px is not None:
20        cfg["min_length_px"] = min_length_px
21    small, scale = CrackAutoDetector._downsample(arr)
22    width_contrast = CrackAutoDetector._width_contrast(arr)
23    contrast = CrackAutoDetector._local_dark_contrast(small)
24    strong_mask = CrackAutoDetector._build_mask(small, contrast, cfg
25    if sensitivity == "增强":
26        strong_mask = CrackAutoDetector._dilate(strong_mask)
27        strong_mask = CrackAutoDetector._neighbor_filter(strong_mask
28    comps = CrackAutoDetector._components(strong_mask)
29    scored: list[tuple[float, CrackObject, tuple[float, float, float
30    used_boxes: list[tuple[float, float, float, float]] = []
31    for comp in sorted(comps, key=len, reverse=True):
32        if len(comp) < cfg["min_area"]:
33            continue
34        pts_small = np.array(comp, dtype=float)
35        if not CrackAutoDetector._looks_like_crack(pts_small, strong
36            continue
37        pts = np.array([(x * scale, y * scale) for x, y in comp], dt
38        poly = CrackAutoDetector._fit_polyline(pts)
39        if len(poly) < 2:
40            continue
41        length = polyline_length(poly)
42        if length < cfg["min_length_px"]:
43            continue
44        support = CrackAutoDetector._line_support_score(poly, strong
45        if support < cfg["support"]:
46            continue
47        box = CrackAutoDetector._bbox(poly)
48        if CrackAutoDetector._overlaps_existing(box, used_boxes, thr
49            continue
50        crack = CrackObject(points=poly, source="自动识别")
51

```

```

1         CrackAutoDetector._auto_fill_widths_from_contrast(width_cont
2         CrackSeverityEvaluator.refresh_crack(crack, meter_per_pixel)
3         width_bonus = len(crack.width_samples) * 15
4         score = length * 0.35 + support * 180 + width_bonus
5         scored.append((score, crack, box))
6     scored.sort(key=lambda x: x[0], reverse=True)
7     cracks: list[CrackObject] = []
8     for _, crack, box in scored:
9         if CrackAutoDetector._overlaps_existing(box, used_boxes, thr
10             continue
11         used_boxes.append(box)
12         cracks.append(crack)
13         if len(cracks) >= cfg["max_count"]:
14             break
15     return cracks
16 @staticmethod
17 def estimate_widths(image_path: str | Path, crack: CrackObject, mete
18     """返回最窄、中间位置、最宽三处宽度。"""
19     arr = CrackAutoDetector._load_gray(image_path)
20     if arr is None:
21         return []
22     contrast = CrackAutoDetector._width_contrast(arr)
23     samples = CrackAutoDetector._sample_width_profile_from_contrast(
24         return CrackAutoDetector._pick_min_mid_max(samples)
25 @staticmethod
26 def width_summary(samples: list[WidthSample]) -> str:
27     if not samples:
28         return "尚未估算宽度"
29     vals = [s.width_mm for s in samples]
30     return f"最窄: {min(vals):.2f}mm | 中间: {vals[len(vals)//2]:.2f}mm | 最宽: {
31 @staticmethod
32 def _load_gray(image_path: str | Path):
33     try:
34         img = Image.open(image_path).convert("L")
35         return np.array(img)
36     except Exception:
37         return None
38 @staticmethod
39 def _downsample(arr: np.ndarray):
40     h, w = arr.shape
41     scale = max(1, math.ceil(max(h, w) / 980))
42     if scale == 1:
43         return arr, 1
44     hh, ww = h // scale, w // scale
45     arr = arr[:hh * scale, :ww * scale]
46     small = arr.reshape(hh, scale, ww, scale).mean(axis=(1, 3)).astype
47     return small, scale
48 @staticmethod
49 def _local_dark_contrast(gray: np.ndarray) -> np.ndarray:
50     radius = max(7, int(min(gray.shape) / 75))

```

```

1         bg = np.array(Image.fromarray(gray).filter(ImageFilter.GaussianB
2         g = gray.astype(np.int16)
3         contrast = bg - g
4         contrast[contrast < 0] = 0
5         return contrast.astype(np.uint8)
6     @staticmethod
7     def _build_mask(gray: np.ndarray, contrast: np.ndarray, cfg: dict) -
8         c_thr = max(cfg["min_contrast"], int(np.percentile(contrast, cfg
9         dark_cap = int(np.percentile(gray, 70))
10        mask = (contrast >= c_thr) & (gray <= dark_cap)
11        mask = CrackAutoDetector._neighbor_filter(mask, min_neighbors=cf
12        h, w = mask.shape
13        margin_y = max(2, int(h * cfg["edge_guard"]))
14        margin_x = max(2, int(w * cfg["edge_guard"]))
15        mask[:margin_y, :] = False
16        mask[-margin_y:, :] = False
17        mask[:, :margin_x] = False
18        mask[:, -margin_x:] = False
19        return mask
20    @staticmethod
21    def _neighbor_filter(mask: np.ndarray, min_neighbors: int = 2) -> np
22        padded = np.pad(mask, 1, mode="constant", constant_values=False)
23        count = np.zeros_like(mask, dtype=np.uint8)
24        for dy in range(3):
25            for dx in range(3):
26                if dy == 1 and dx == 1:
27                    continue
28                count += padded[dy:dy + mask.shape[0], dx:dx + mask.shap
29        return mask & (count >= min_neighbors)
30    @staticmethod
31    def _dilate(mask: np.ndarray) -> np.ndarray:
32        out = mask.copy()
33        padded = np.pad(mask, 1, mode="constant", constant_values=False)
34        for dy in range(3):
35            for dx in range(3):
36                out |= padded[dy:dy + mask.shape[0], dx:dx + mask.shape[
37        return out
38    @staticmethod
39    def _components(mask: np.ndarray):
40        h, w = mask.shape
41        visited = np.zeros_like(mask, dtype=bool)
42        comps = []
43        for y in range(h):
44            for x in range(w):
45                if not mask[y, x] or visited[y, x]:
46                    continue
47                stack = [(x, y)]
48                visited[y, x] = True
49                comp = []
50                while stack:

```

```

1         cx, cy = stack.pop()
2         comp.append((cx, cy))
3         for nx in (cx - 1, cx, cx + 1):
4             for ny in (cy - 1, cy, cy + 1):
5                 if nx < 0 or ny < 0 or nx >= w or ny >= h:
6                     continue
7                 if visited[ny, nx] or not mask[ny, nx]:
8                     continue
9                 visited[ny, nx] = True
10                stack.append((nx, ny))
11            comps.append(comp)
12    return comps
13
14    @staticmethod
15    def _looks_like_crack(points_small: np.ndarray, shape, cfg: dict) ->
16        if len(points_small) < cfg["min_area"]:
17            return False
18        h, w = shape
19        xs = points_small[:, 0]
20        ys = points_small[:, 1]
21        minx, maxx = float(xs.min()), float(xs.max())
22        miny, maxy = float(ys.min()), float(ys.max())
23        span_x = maxx - minx + 1
24        span_y = maxy - miny + 1
25        major_span = max(span_x, span_y)
26        minor_span = min(span_x, span_y)
27        area = float(len(points_small))
28        bbox_area = max(1.0, span_x * span_y)
29        fill_ratio = area / bbox_area
30        if major_span < max(18, cfg["min_length_px"] / 5):
31            return False
32        if fill_ratio > 0.42 and area > 35:
33            return False
34        center = points_small.mean(axis=0)
35        shifted = points_small - center
36        cov = np.cov(shifted.T)
37        vals, _ = np.linalg.eigh(cov)
38        vals = np.sort(vals)[::-1]
39        if vals[0] <= 1e-6:
40            return False
41        slenderness = vals[0] / max(vals[1], 1e-6)
42        if slenderness < 1.8 and minor_span > major_span * 0.65 and area
43            return False
44        edge_band_x = w * 0.06
45        edge_band_y = h * 0.06
46        near_edge = minx < edge_band_x or maxx > w - edge_band_x or miny
47        if near_edge and major_span < max(36, min(w, h) * 0.08):
48            return False
49        return True
50
51    @staticmethod
52    def _fit_polyline(points: np.ndarray) -> List[Point]:

```

```

1     if len(points) < 2:
2         return []
3     center = points.mean(axis=0)
4     shifted = points - center
5     cov = np.cov(shifted.T)
6     vals, vecs = np.linalg.eigh(cov)
7     order = np.argsort(vals)[::-1]
8     vecs = vecs[:, order]
9     major = vecs[:, 0]
10    proj_major = shifted @ major
11    major_span = proj_major.max() - proj_major.min()
12    if major_span < 20:
13        return []
14    bin_count = 6 if major_span < 160 else 8
15    bins = np.linspace(proj_major.min(), proj_major.max(), bin_count)
16    result = []
17    for i in range(len(bins) - 1):
18        mask = (proj_major >= bins[i]) & (proj_major <= bins[i + 1])
19        sel = points[mask]
20        if len(sel) == 0:
21            continue
22        x, y = np.median(sel[:, 0]), np.median(sel[:, 1])
23        if not result or distance(result[-1], (float(x), float(y)))
24            result.append((float(x), float(y)))
25    if len(result) < 2:
26        a = center + major * proj_major.min()
27        b = center + major * proj_major.max()
28        result = [(float(a[0]), float(a[1])), (float(b[0]), float(b[
29    return result
30
31    @staticmethod
32    def _line_support_score(poly: list[Point], mask_small: np.ndarray, s
33        h, w = mask_small.shape
34        hit = 0
35        total = 0
36        for a, b in zip(poly[:-1], poly[1:]):
37            ax, ay = a[0] / scale, a[1] / scale
38            bx, by = b[0] / scale, b[1] / scale
39            seg_len = max(1, int(math.hypot(bx - ax, by - ay)))
40            steps = max(8, min(80, seg_len))
41            for i in range(steps + 1):
42                t = i / steps
43                x = int(round(ax + (bx - ax) * t))
44                y = int(round(ay + (by - ay) * t))
45                if x < 0 or y < 0 or x >= w or y >= h:
46                    continue
47                total += 1
48                x1, x2 = max(0, x - width), min(w, x + width + 1)
49                y1, y2 = max(0, y - width), min(h, y + width + 1)
50                if mask_small[y1:y2, x1:x2].any():
51                    hit += 1

```

```

1         return hit / max(1, total)
2     @staticmethod
3     def _width_contrast(arr: np.ndarray) -> np.ndarray:
4         radius = max(5, int(min(arr.shape) / 90))
5         bg = np.array(Image.fromarray(arr).filter(ImageFilter.GaussianBl
6         gray = arr.astype(np.int16)
7         contrast = bg - gray
8         contrast[contrast < 0] = 0
9         return contrast.astype(np.uint8)
10    @staticmethod
11    def _auto_fill_widths_from_contrast(contrast: np.ndarray, crack: Cra
12        samples = CrackAutoDetector._sample_width_profile_from_contrast(
13        crack.width_samples = CrackAutoDetector._pick_min_mid_max(sample
14    @staticmethod
15    def _sample_width_profile_from_contrast(contrast: np.ndarray, points
16        if len(points) < 2:
17            return []
18        local_thr = max(10, int(np.percentile(contrast, 91)))
19        samples: list[WidthSample] = []
20        for idx in range(len(points) - 1):
21            p1 = np.array(points[idx], dtype=float)
22            p2 = np.array(points[idx + 1], dtype=float)
23            v = p2 - p1
24            n = np.linalg.norm(v)
25            if n < 1e-6:
26                continue
27            dirv = v / n
28            normal = np.array([-dirv[1], dirv[0]])
29            center = (p1 + p2) / 2
30            left = CrackAutoDetector._trace_edge(contrast, center, -norm
31            right = CrackAutoDetector._trace_edge(contrast, center, norm
32            if left is None or right is None:
33                continue
34            a = tuple(left.tolist())
35            b = tuple(right.tolist())
36            width_px = distance(a, b)
37            if width_px < 1.2 or width_px > 55:
38                continue
39            width_mm = width_px * meter_per_pixel * 1000 if meter_per_pi
40            samples.append(WidthSample(p1=a, p2=b, width_mm=width_mm))
41        return samples
42    @staticmethod
43    def _pick_min_mid_max(samples: list[WidthSample]) -> list[WidthSampl
44        if not samples:
45            return []
46        min_s = min(samples, key=lambda s: s.width_mm)
47        mid_s = samples[len(samples) // 2]
48        max_s = max(samples, key=lambda s: s.width_mm)
49        return [min_s, mid_s, max_s]
50    @staticmethod

```

```

1  def _trace_edge(contrast: np.ndarray, center: np.ndarray, direction:
2      h, w = contrast.shape
3      last_line = None
4      for step in range(1, 30):
5          p = center + direction * step
6          x, y = int(round(p[0])), int(round(p[1]))
7          if x < 0 or y < 0 or x >= w or y >= h:
8              break
9          val = contrast[y, x]
10         if val >= thr:
11             last_line = np.array([x, y], dtype=float)
12         elif last_line is not None:
13             return last_line
14     return last_line
15 @staticmethod
16 def _bbox(points: list[Point]) -> tuple[float, float, float, float]:
17     xs = [p[0] for p in points]
18     ys = [p[1] for p in points]
19     return min(xs), min(ys), max(xs), max(ys)
20 @staticmethod
21 def _overlaps_existing(box, boxes, threshold: float = 0.65) -> bool:
22     x1, y1, x2, y2 = box
23     area = max(1.0, (x2 - x1) * (y2 - y1))
24     for bx1, by1, bx2, by2 in boxes:
25         ix1, iy1 = max(x1, bx1), max(y1, by1)
26         ix2, iy2 = min(x2, bx2), min(y2, by2)
27         if ix2 <= ix1 or iy2 <= iy1:
28             continue
29         inter = (ix2 - ix1) * (iy2 - iy1)
30         if inter / area > threshold:
31             return True
32     return False
33 from __future__ import annotations
34 from typing import Tuple
35 from .geometry import distance
36 Point = Tuple[float, float]
37 class ImageCalibrationEngine:
38     """比例尺标定引擎：把像素长度转换为真实道路长度。"""
39     @staticmethod
40     def calibrate_by_two_points(p1: Point, p2: Point, real_distance_m: f
41         pixel_distance = distance(p1, p2)
42         if pixel_distance <= 0:
43             raise ValueError("两个标定点距离过小，无法计算比例尺")
44         if real_distance_m <= 0:
45             raise ValueError("实际距离必须大于 0")
46         return real_distance_m / pixel_distance
47     @staticmethod
48     def describe(meter_per_pixel: float | None) -> str:
49         if not meter_per_pixel:
50             return "未标定：当前量测结果以像素为单位"

```



```

1         return f"已标定: 1 像素约等于 {meter_per_pixel * 1000:.3f} 毫米"
2     @staticmethod
3     def pixel_to_meter(pixel: float, meter_per_pixel: float | None) -> f
4         return pixel * meter_per_pixel if meter_per_pixel else pixel
5     @staticmethod
6     def pixel_to_mm(pixel: float, meter_per_pixel: float | None) -> floa
7         return pixel * meter_per_pixel * 1000 if meter_per_pixel else pi
8 from __future__ import annotations
9 import math
10 from typing import List, Tuple
11 Point = Tuple[float, float]
12 def distance(p1: Point, p2: Point) -> float:
13     return math.hypot(p1[0] - p2[0], p1[1] - p2[1])
14 def polyline_length(points: List[Point]) -> float:
15     return sum(distance(points[i - 1], points[i]) for i in range(1, len(
16 def polyline_length_real(points: List[Point], meter_per_pixel: float | N
17     if not meter_per_pixel:
18         return polyline_length(points)
19     return polyline_length(points) * meter_per_pixel
20 def polygon_area_pixels(points: List[Point]) -> float:
21     if len(points) < 3:
22         return 0.0
23     area = 0.0
24     for i, (x1, y1) in enumerate(points):
25         x2, y2 = points[(i + 1) % len(points)]
26         area += x1 * y2 - x2 * y1
27     return abs(area) / 2.0
28 def polygon_area_real(points: List[Point], meter_per_pixel: float | None
29     area_px = polygon_area_pixels(points)
30     if not meter_per_pixel:
31         return area_px
32     return area_px * (meter_per_pixel ** 2)
33 def smooth_polyline(points: List[Point], strength: float = 0.55, rounds:
34     if len(points) <= 2:
35         return list(points)
36     result = list(points)
37     for _ in range(rounds):
38         new_points = [result[0]]
39         for i in range(1, len(result) - 1):
40             px, py = result[i - 1]
41             x, y = result[i]
42             nx, ny = result[i + 1]
43             sx = x * (1 - strength) + (px + nx) / 2 * strength
44             sy = y * (1 - strength) + (py + ny) / 2 * strength
45             new_points.append((sx, sy))
46         new_points.append(result[-1])
47         result = new_points
48     return result
49 def bounding_box(points: List[Point]) -> tuple[float, float, float, floa
50     if not points:

```

```

1         return 0, 0, 0, 0
2     xs, ys = [p[0] for p in points], [p[1] for p in points]
3     return min(xs), min(ys), max(xs), max(ys)
4 def nearest_point_index(points: List[Point], p: Point, radius: float = 1
5     best = -1
6     best_d = radius
7     for i, q in enumerate(points):
8         d = distance(p, q)
9         if d < best_d:
10             best_d = d
11             best = i
12     return best
13 from __future__ import annotations
14 from pathlib import Path
15 from PyQt6.QtGui import QImage, QPainter, QColor, QPen
16 from PyQt6.QtCore import QPointF, Qt
17 def ensure_sample_images(sample_dir: Path):
18     sample_dir.mkdir(parents=True, exist_ok=True)
19     if list(sample_dir.glob('*.png')):
20         return
21     for index in range(1, 3):
22         create_fallback_sample(sample_dir / f'fallback_sample_{index}.pn
23 def create_fallback_sample(path: Path, seed: int):
24     w, h = 1280, 820
25     image = QImage(w, h, QImage.Format.Format_RGB32)
26     image.fill(QColor(144, 148, 150))
27     painter = QPainter(image)
28     painter.setRenderHint(QPainter.RenderHint.Antialiasing)
29     for i in range(2200):
30         x = (i * 97 + seed * 31) % w
31         y = (i * 53 + seed * 71) % h
32         g = 120 + ((i * 17) % 45)
33         painter.setPen(QPen(QColor(g, g, g), 1))
34         painter.drawPoint(x, y)
35     painter.setPen(QPen(QColor(40, 40, 40), 10, Qt.PenStyle.SolidLine, Q
36     cracks = [
37         [(160, 190), (320, 240), (450, 280), (620, 330), (920, 380)],
38         [(740, 90), (710, 210), (760, 360), (720, 520), (770, 710)],
39     ]
40     for line in cracks:
41         for a, b in zip(line[:-1], line[1:]):
42             painter.drawLine(QPointF(*a), QPointF(*b))
43     painter.end()
44     image.save(str(path))
45 from __future__ import annotations
46 import math
47 from typing import List
48 from .models import CandidateCrack
49 from .geometry import distance
50 class MissingCrackScanner:

```

```

1      """疑似漏标扫描：基于暗像素扫描生成候选裂缝线。演示版不依赖 OpenCV。"""
2      @staticmethod
3      def scan_qimage(qimage, existing_cracks=None, max_candidates: int =
4          existing_cracks = existing_cracks or []
5          if qimage is None or qimage.isNull():
6              return []
7          w, h = qimage.width(), qimage.height()
8          step_y = max(12, h // 40)
9          step_x = max(10, w // 60)
10         raw_segments = []
11         for y in range(step_y, h - step_y, step_y):
12             dark_run = []
13             for x in range(step_x, w - step_x, step_x):
14                 c = qimage.pixelColor(x, y)
15                 gray = (c.red() * 30 + c.green() * 59 + c.blue() * 11) /
16                 if gray < 88:
17                     dark_run.append((x, y + math.sin(x / 20) * 2))
18                 else:
19                     if len(dark_run) >= 3:
20                         raw_segments.append(dark_run)
21                         dark_run = []
22             if len(dark_run) >= 3:
23                 raw_segments.append(dark_run)
24         candidates = []
25         for seg in raw_segments:
26             if MissingCrackScanner._covered_by_existing(seg, existing_cr
27                 continue
28             score = min(99, 45 + len(seg) * 6)
29             candidates.append(CandidateCrack(points=seg, score=score))
30             if len(candidates) >= max_candidates:
31                 break
32         return candidates
33     @staticmethod
34     def _covered_by_existing(points, cracks) -> bool:
35         for p in points:
36             for crack in cracks:
37                 for q in crack.points:
38                     if distance(p, q) < 20:
39                         return True
40         return False
41     from __future__ import annotations
42     from dataclasses import dataclass, field, asdict
43     from datetime import datetime
44     from typing import List, Optional, Tuple, Dict, Any
45     import uuid
46     Point = Tuple[float, float]
47     def new_id(prefix: str) -> str:
48         return f"{prefix}-{uuid.uuid4().hex[:8]}"
49     @dataclass
50     class User:

```

```

1     username: str
2     password_hash: str
3     role: str = "标注员"
4     created_at: str = field(default_factory=lambda: datetime.now().strftime
5     def to_dict(self) -> Dict[str, Any]:
6         return asdict(self)
7     @staticmethod
8     def from_dict(data: Dict[str, Any]) -> "User":
9         return User(
10             username=data.get("username", ""),
11             password_hash=data.get("password_hash", ""),
12             role=data.get("role", "标注员"),
13             created_at=data.get("created_at", "") or datetime.now().strftime
14         )
15 @dataclass
16 class WidthSample:
17     p1: Point
18     p2: Point
19     width_mm: float
20     created_at: str = field(default_factory=lambda: datetime.now().strftime
21     def to_dict(self) -> Dict[str, Any]:
22         return {"p1": list(self.p1), "p2": list(self.p2), "width_mm": se
23     @staticmethod
24     def from_dict(data: Dict[str, Any]) -> "WidthSample":
25         return WidthSample(tuple(data.get("p1", (0, 0))), tuple(data.get
26 @dataclass
27 class ReviewLog:
28     crack_id: str
29     reviewer: str
30     result: str
31     comment: str
32     created_at: str = field(default_factory=lambda: datetime.now().strftime
33     def to_dict(self) -> Dict[str, Any]:
34         return asdict(self)
35     @staticmethod
36     def from_dict(data: Dict[str, Any]) -> "ReviewLog":
37         return ReviewLog(
38             crack_id=data.get("crack_id", ""),
39             reviewer=data.get("reviewer", ""),
40             result=data.get("result", ""),
41             comment=data.get("comment", ""),
42             created_at=data.get("created_at", ""),
43         )
44 @dataclass
45 class CrackObject:
46     crack_id: str = field(default_factory=lambda: new_id("裂缝"))
47     points: List[Point] = field(default_factory=list)
48     crack_type: str = "待判定"
49     suggestion_type: str = "待判定"
50     severity: str = "未评估"
51

```

```

1 review_status: str = "待复核"
2 review_comment: str = ""
3 locked: bool = False
4 ignored: bool = False
5 source: str = "人工描绘"
6 width_samples: List[WidthSample] = field(default_factory=list)
7 repair_method: str = "灌缝胶修补"
8 material_amount: float = 0.0
9 material_unit: str = "升"
10 station: str = "未绑定"
11 created_at: str = field(default_factory=lambda: datetime.now().strftime
12 @property
13 def avg_width_mm(self) -> float:
14     if not self.width_samples:
15         return 0.0
16     return sum(s.width_mm for s in self.width_samples) / len(self.wi
17 @property
18 def max_width_mm(self) -> float:
19     if not self.width_samples:
20         return 0.0
21     return max(s.width_mm for s in self.width_samples)
22 def to_dict(self) -> Dict[str, Any]:
23     data = asdict(self)
24     data["points"] = [list(p) for p in self.points]
25     data["width_samples"] = [s.to_dict() for s in self.width_samples]
26     return data
27 @staticmethod
28 def from_dict(data: Dict[str, Any]) -> "CrackObject":
29     crack = CrackObject(
30         crack_id=data.get("crack_id") or new_id("裂缝"),
31         points=[tuple(p) for p in data.get("points", [])],
32         crack_type=data.get("crack_type", "待判定"),
33         suggestion_type=data.get("suggestion_type", "待判定"),
34         severity=data.get("severity", "未评估"),
35         review_status=data.get("review_status", "待复核"),
36         review_comment=data.get("review_comment", ""),
37         locked=bool(data.get("locked", False)),
38         ignored=bool(data.get("ignored", False)),
39         source=data.get("source", "人工描绘"),
40         repair_method=data.get("repair_method", "灌缝胶修补"),
41         material_amount=float(data.get("material_amount", 0)),
42         material_unit=data.get("material_unit", "升"),
43         station=data.get("station", "未绑定"),
44         created_at=data.get("created_at", ""),
45     )
46     crack.width_samples = [WidthSample.from_dict(x) for x in data.ge
47     return crack
48 @dataclass
49 class CrackRegion:
50     region_id: str = field(default_factory=lambda: new_id("网裂区"))
51

```

```

1 polygon_points: List[Point] = field(default_factory=list)
2 area_m2: float = 0.0
3 density: float = 0.0
4 severity: str = "未评估"
5 review_status: str = "待复核"
6 comment: str = ""
7 def to_dict(self) -> Dict[str, Any]:
8     data = asdict(self)
9     data["polygon_points"] = [list(p) for p in self.polygon_points]
10    return data
11 @staticmethod
12 def from_dict(data: Dict[str, Any]) -> "CrackRegion":
13     return CrackRegion(
14         region_id=data.get("region_id") or new_id("网裂区"),
15         polygon_points=[tuple(p) for p in data.get("polygon_points",
16         area_m2=float(data.get("area_m2", 0)),
17         density=float(data.get("density", 0)),
18         severity=data.get("severity", "未评估"),
19         review_status=data.get("review_status", "待复核"),
20         comment=data.get("comment", ""),
21     )
22 @dataclass
23 class CandidateCrack:
24     candidate_id: str = field(default_factory=lambda: new_id("疑似"))
25     points: List[Point] = field(default_factory=list)
26     score: float = 0.0
27     status: str = "待确认"
28     note: str = "暗线增强候选"
29     def to_dict(self) -> Dict[str, Any]:
30         return {"candidate_id": self.candidate_id, "points": [list(p) fo
31 @staticmethod
32 def from_dict(data: Dict[str, Any]) -> "CandidateCrack":
33     return CandidateCrack(data.get("candidate_id") or new_id("疑似"),
34 @dataclass
35 class AnnotationTask:
36     task_id: str = field(default_factory=lambda: new_id("任务"))
37     road_name: str = "未命名道路"
38     image_path: str = ""
39     pavement_type: str = "沥青路面"
40     station_start: str = "K0+000"
41     station_end: str = "K0+100"
42     lane: str = "上行一车道"
43     location_name: str = "未注明位置"
44     longitude: str = ""
45     latitude: str = ""
46     road_level: str = "一般公路"
47     direction: str = "上行"
48     section_note: str = ""
49     source_type: str = "普通路面影像"
50 collector: str = "演示采集员"
51

```

```

1  annotator: str = ""
2  status: str = "待标注"
3  meter_per_pixel: Optional[float] = None
4  cracks: List[CrackObject] = field(default_factory=list)
5  regions: List[CrackRegion] = field(default_factory=list)
6  candidates: List[CandidateCrack] = field(default_factory=list)
7  review_logs: List[ReviewLog] = field(default_factory=list)
8  created_at: str = field(default_factory=lambda: datetime.now().strftime
9  updated_at: str = field(default_factory=lambda: datetime.now().strftime
10 def touch(self):
11     self.updated_at = datetime.now().strftime("%Y-%m-%d %H:%M:%S")
12 def to_dict(self) -> Dict[str, Any]:
13     data = asdict(self)
14     data["cracks"] = [c.to_dict() for c in self.cracks]
15     data["regions"] = [r.to_dict() for r in self.regions]
16     data["candidates"] = [c.to_dict() for c in self.candidates]
17     data["review_logs"] = [r.to_dict() for r in self.review_logs]
18     return data
19 @staticmethod
20 def from_dict(data: Dict[str, Any]) -> "AnnotationTask":
21     task = AnnotationTask(
22         task_id=data.get("task_id") or new_id("任务"),
23         road_name=data.get("road_name", "未命名道路"),
24         image_path=data.get("image_path", ""),
25         pavement_type=data.get("pavement_type", "沥青路面"),
26         station_start=data.get("station_start", "K0+000"),
27         station_end=data.get("station_end", "K0+100"),
28         lane=data.get("lane", "上行一车道"),
29         location_name=data.get("location_name", "未注明位置"),
30         longitude=data.get("longitude", ""),
31         latitude=data.get("latitude", ""),
32         road_level=data.get("road_level", "一般公路"),
33         direction=data.get("direction", "上行"),
34         section_note=data.get("section_note", ""),
35         source_type=data.get("source_type", "普通路面影像"),
36         collector=data.get("collector", "演示采集员"),
37         annotator=data.get("annotator", ""),
38         status=data.get("status", "待标注"),
39         meter_per_pixel=data.get("meter_per_pixel", None),
40         created_at=data.get("created_at", ""),
41         updated_at=data.get("updated_at", ""),
42     )
43     task.cracks = [CrackObject.from_dict(x) for x in data.get("crack
44     task.regions = [CrackRegion.from_dict(x) for x in data.get("regi
45     task.candidates = [CandidateCrack.from_dict(x) for x in data.get
46     task.review_logs = [ReviewLog.from_dict(x) for x in data.get("re
47     return task
48 from __future__ import annotations
49 class AnnotationQualityChecker:
50     """标注质量评分：用于检查比例尺、宽度采样、复核状态、漏标候选等问题。"""
51

```

```

1  @staticmethod
2  def score(task):
3      score = 100
4      details = []
5      if not task:
6          return 0, ["未创建标注任务"]
7      if not task.meter_per_pixel:
8          score -= 20
9          details.append("未完成比例尺标定, 扣 20 分")
10     if not task.cracks and not task.regions:
11         score -= 25
12         details.append("当前影像没有正式裂缝或网裂区域, 扣 25 分")
13     for crack in task.cracks:
14         if len(crack.points) < 2:
15             score -= 5
16             details.append(f"{crack.crack_id} 节点不足, 扣 5 分")
17         if crack.crack_type in ["", "待判定", "未分类"]:
18             score -= 5
19             details.append(f"{crack.crack_id} 类型未确认, 扣 5 分")
20         if not crack.width_samples:
21             score -= 4
22             details.append(f"{crack.crack_id} 未进行宽度采样, 扣 4 分")
23         if crack.review_status not in ["已通过", "已锁定"]:
24             score -= 3
25             details.append(f"{crack.crack_id} 未复核通过, 扣 3 分")
26     active_candidates = [c for c in task.candidates if c.status == ""]
27     if active_candidates:
28         score -= min(15, len(active_candidates) * 3)
29         details.append(f"仍有 {len(active_candidates)} 条疑似漏标候选未处理")
30     score = max(0, min(100, int(score)))
31     if not details:
32         details.append("标注完整性较好, 可进入成果导出")
33     return score, details
34 from __future__ import annotations
35 from typing import List, Tuple
36 from .geometry import polygon_area_real
37 Point = Tuple[float, float]
38 class CrackRegionAnalyzer:
39     """网裂区域分析: 计算圈选面积和裂缝密度。"""
40     @staticmethod
41     def area(points: List[Point], meter_per_pixel: float | None) -> float:
42         return polygon_area_real(points, meter_per_pixel)
43     @staticmethod
44     def density(total_length_m: float, area_m2: float) -> float:
45         if area_m2 <= 0:
46             return 0.0
47         return total_length_m / area_m2
48     @staticmethod
49     def severity(area_m2: float, density: float) -> str:
50         score = 0
51

```



```

1         if area_m2 > 20:
2             score += 2
3         elif area_m2 > 5:
4             score += 1
5         if density > 3:
6             score += 2
7         elif density > 1:
8             score += 1
9         if score <= 1:
10            return "轻微"
11        if score <= 2:
12            return "一般"
13        if score <= 3:
14            return "较重"
15        return "严重"
16    from __future__ import annotations
17    class RepairMaterialEstimator:
18        """修复材料估算：把裂缝量测结果转换为养护材料建议。"""
19        METHODS = ["灌缝胶修补", "开槽灌缝", "沥青冷补料", "局部铣刨重铺", "裂缝封层", "网裂区域罩面"]
20        @staticmethod
21        def estimate_sealant(length_m: float, avg_width_mm: float, depth_mm:
22            width_m = max(avg_width_mm, 1.2) / 1000
23            depth_m = depth_mm / 1000
24            volume_m3 = length_m * width_m * depth_m
25            return volume_m3 * 1000 * (1 + loss_rate)
26        @staticmethod
27        def estimate_patch_material(area_m2: float, thickness_cm: float = 4,
28            volume_m3 = area_m2 * (thickness_cm / 100)
29            return volume_m3 * density_ton_m3 * (1 + loss_rate)
30        @staticmethod
31        def recommend(severity: str, crack_type: str) -> str:
32            if crack_type in ["网状裂缝", "龟裂", "块状裂缝"]:
33                return "网裂区域罩面" if severity in ["较重", "严重"] else "裂缝封层"
34            if severity in ["轻微", "一般"]:
35                return "灌缝胶修补"
36            if severity == "较重":
37                return "开槽灌缝"
38            return "局部铣刨重铺"
39        @staticmethod
40        def estimate_for_crack(length_m: float, avg_width_mm: float, severit
41            chosen = method or RepairMaterialEstimator.recommend(severity, c
42            if chosen == "灌缝胶修补":
43                return chosen, RepairMaterialEstimator.estimate_sealant(leng
44            if chosen == "开槽灌缝":
45                return chosen, RepairMaterialEstimator.estimate_sealant(leng
46            if chosen == "裂缝封层":
47                return chosen, max(length_m * 0.25, 0.5), "平方米"
48            if chosen == "局部铣刨重铺":
49                return chosen, max(length_m * 0.8, 1.0) * 0.04 * 2.3 * 1.12,
50            if chosen == "沥青冷补料":

```

```

1         return chosen, max(length_m * 0.15, 0.2), "吨"
2         return chosen, max(length_m * 0.6, 0.8), "平方米"
3 from __future__ import annotations
4 import csv
5 import json
6 from pathlib import Path
7 from datetime import datetime
8 from .geometry import polyline_length_real
9 from .statistics import RoadSectionStatistics
10 from .quality import AnnotationQualityChecker
11 class CrackReportGenerator:
12     """成果导出: JSON、CSV、TXT 报告。"""
13     def __init__(self, export_dir: Path):
14         self.export_dir = Path(export_dir)
15         self.export_dir.mkdir(parents=True, exist_ok=True)
16     def export_json(self, tasks):
17         path = self.export_dir / f"crack_annotations_{datetime.now().str
18         data = [task.to_dict() for task in tasks]
19         path.write_text(json.dumps(data, ensure_ascii=False, indent=2),
20         return path
21     def export_csv(self, tasks):
22         path = self.export_dir / f"crack_measurements_{datetime.now().st
23         with path.open("w", newline="", encoding="utf-8-sig") as f:
24             writer = csv.writer(f)
25             writer.writerow(["任务编号", "道路名称", "位置说明", "经度", "纬度", "方向", "
26             for task in tasks:
27                 for crack in task.cracks:
28                     length = polyline_length_real(crack.points, task.met
29                     if not task.meter_per_pixel:
30                         length = length / 180
31                     writer.writerow([task.task_id, task.road_name, task.
32         return path
33     def export_txt_report(self, tasks):
34         path = self.export_dir / f"road_crack_report_{datetime.now().str
35         stat = RoadSectionStatistics.summarize_tasks(tasks)
36         lines = []
37         lines.append("路面裂缝人工标注与测量分析报告")
38         lines.append("=" * 36)
39         lines.append(f"生成时间: {datetime.now().strftime('%Y-%m-%d %H:%M:%S'
40         lines.append(f"任务数量: {len(tasks)}")
41         lines.append("")
42         lines.append("一、路段病害汇总")
43         lines.append(f"裂缝总数: {stat['裂缝总数']} 条")
44         lines.append(f"网裂区域数: {stat['网裂区域数']} 个")
45         lines.append(f"裂缝总长度: {stat['裂缝总长度']:.2f} 米")
46         lines.append(f"网裂总面积: {stat['网裂总面积']:.2f} 平方米")
47         lines.append(f"平均宽度: {stat['平均宽度']:.2f} 毫米")
48         lines.append(f"较重及严重数量: {stat['较重及严重数量']} 个")
49         lines.append(f"病害指数: {stat['病害指数']} / 100")
50         lines.append("")

```

```

1     lines.append("二、类型分布")
2     for k, v in stat["类型分布"].items():
3         lines.append(f"{k}: {v}")
4     lines.append("")
5     lines.append("三、严重等级分布")
6     for k, v in stat["等级分布"].items():
7         lines.append(f"{k}: {v}")
8     lines.append("")
9     lines.append("四、任务明细与质量检查")
10    for task in tasks:
11        score, details = AnnotationQualityChecker.score(task)
12        lines.append(f"任务: {task.road_name} / {task.task_id} / 质量分: {s")
13        lines.append(f"影像: {task.image_path}")
14        lines.append(f"位置说明: {task.location_name}")
15        lines.append(f"坐标: {task.latitude}, {task.longitude}")
16        lines.append(f"方向/等级: {task.direction} / {task.road_level}")
17        lines.append(f"影像来源: {task.source_type}")
18        lines.append(f"路段备注: {task.section_note}")
19        lines.append(f"比例尺: {task.meter_per_pixel} if task.meter_per_p
20    for d in details[:8]:
21        lines.append(f"- {d}")
22    lines.append("")
23    lines.append("五、养护建议")
24    if stat["病害指数"] >= 70:
25        lines.append("该路段病害指数较高, 建议优先安排现场复核, 并对严重裂缝实施开槽灌缝或局部铣刨重铺。")
26    elif stat["病害指数"] >= 40:
27        lines.append("该路段存在较明显裂缝病害, 建议近期进行灌缝、封层和复核性检测。")
28    else:
29        lines.append("该路段当前病害指数较低, 可纳入日常巡查和预防性养护计划。")
30    path.write_text("\n".join(lines), encoding="utf-8")
31    return path
32
33 from __future__ import annotations
34 from typing import List, Tuple
35 from .geometry import bounding_box, polyline_length_real
36 Point = Tuple[float, float]
37 class CrackSeverityEvaluator:
38     """裂缝诊断: 类型建议、严重等级与判定依据。"""
39     @staticmethod
40     def classify(points: List[Point], is_region: bool = False) -> str:
41         if is_region:
42             return "网状裂缝"
43         if len(points) < 2:
44             return "待判定"
45         minx, miny, maxx, maxy = bounding_box(points)
46         span_x = maxx - minx
47         span_y = maxy - miny
48         if span_x > span_y * 2:
49             return "横向裂缝"
50         if span_y > span_x * 2:
51             return "纵向裂缝"

```

```

1         if len(points) >= 6 and (span_x + span_y) > 160:
2             return "斜向裂缝"
3         return "斜向裂缝"
4     @staticmethod
5     def evaluate(length_m: float, avg_width_mm: float, max_width_mm: flo
6         score = 0
7         reasons = []
8         if length_m > 5:
9             score += 2
10            reasons.append("裂缝长度超过 5 米")
11        elif length_m > 2:
12            score += 1
13            reasons.append("裂缝长度超过 2 米")
14        else:
15            reasons.append("裂缝长度较短")
16        if avg_width_mm > 5:
17            score += 2
18            reasons.append("平均宽度超过 5 毫米")
19        elif avg_width_mm > 2:
20            score += 1
21            reasons.append("平均宽度超过 2 毫米")
22        else:
23            reasons.append("平均宽度未超过明显阈值")
24        if max_width_mm > 8:
25            score += 2
26            reasons.append("最大宽度超过 8 毫米, 需重点复核")
27        if crack_type in ["网状裂缝", "龟裂", "块状裂缝"]:
28            score += 2
29            reasons.append("病害类型属于面状或结构风险更高的类型")
30        if score <= 1:
31            return "轻微", reasons
32        if score <= 3:
33            return "一般", reasons
34        if score <= 5:
35            return "较重", reasons
36        return "严重", reasons
37    @staticmethod
38    def refresh_crack(crack, meter_per_pixel: float | None):
39        length = polyline_length_real(crack.points, meter_per_pixel)
40        display_length = length if meter_per_pixel else length / 180
41        crack.suggestion_type = CrackSeverityEvaluator.classify(crack.po
42        if crack.crack_type in ["待判定", "", "未分类"]:
43            crack.crack_type = crack.suggestion_type
44        severity, reasons = CrackSeverityEvaluator.evaluate(display_leng
45        crack.severity = severity
46        return reasons
47    from __future__ import annotations
48    from typing import Tuple, Optional
49    Point = Tuple[float, float]
50    class CrackSnapAssistant:
51

```

```

1  """磁吸辅助：在点击点周围搜索更暗的像素，使人工点位靠近裂缝暗线。"""
2  def __init__(self, enabled: bool = False, radius: int = 10):
3      self.enabled = enabled
4      self.radius = radius
5      self.last_message = "磁吸未启用"
6  def set_enabled(self, value: bool):
7      self.enabled = value
8      self.last_message = "磁吸辅助已开启" if value else "磁吸辅助已关闭"
9  def set_radius(self, radius: int):
10     self.radius = max(2, min(int(radius), 40))
11     self.last_message = f"磁吸半径已调整为 {self.radius} 像素"
12  def snap_qimage(self, qimage, x: float, y: float) -> Point:
13     if not self.enabled or qimage is None or qimage.isNull():
14         return (x, y)
15     width, height = qimage.width(), qimage.height()
16     cx, cy = int(round(x)), int(round(y))
17     best = (cx, cy)
18     best_score = 999999
19     r = self.radius
20     for yy in range(max(0, cy - r), min(height, cy + r + 1)):
21         for xx in range(max(0, cx - r), min(width, cx + r + 1)):
22             color = qimage.pixelColor(xx, yy)
23             gray = (color.red() * 30 + color.green() * 59 + color.bl
24             dist_penalty = ((xx - cx) ** 2 + (yy - cy) ** 2) ** 0.5
25             score = gray + dist_penalty
26             if score < best_score:
27                 best_score = score
28                 best = (xx, yy)
29         if abs(best[0] - cx) + abs(best[1] - cy) > 0:
30             self.last_message = f"点位已磁吸: ({cx},{cy}) -> ({best[0]},{best[1]
31         return float(best[0]), float(best[1])
32  from __future__ import annotations
33  from collections import Counter
34  from .geometry import polyline_length_real
35  class RoadSectionStatistics:
36     """路段级统计：把多张影像中的裂缝汇总为路段调查指标。"""
37     @staticmethod
38     def summarize_tasks(tasks):
39         total_cracks = 0
40         total_regions = 0
41         total_length_m = 0.0
42         total_area_m2 = 0.0
43         severe_count = 0
44         type_counter = Counter()
45         level_counter = Counter()
46         width_values = []
47         for task in tasks:
48             mpp = task.meter_per_pixel
49             for crack in task.cracks:
50                 total_cracks += 1

```

```

1         length = polyline_length_real(crack.points, mpp)
2         if not mpp:
3             length = length / 180
4         total_length_m += length
5         type_counter[crack.crack_type] += 1
6         level_counter[crack.severity] += 1
7         if crack.severity in ["严重", "较重"]:
8             severe_count += 1
9         if crack.avg_width_mm:
10            width_values.append(crack.avg_width_mm)
11    for region in task.regions:
12        total_regions += 1
13        total_area_m2 += region.area_m2
14        level_counter[region.severity] += 1
15        if region.severity in ["严重", "较重"]:
16            severe_count += 1
17    avg_width = sum(width_values) / len(width_values) if width_value
18    risk_index = min(100, int(total_length_m * 3 + total_area_m2 * 2
19    return {
20        "裂缝总数": total_cracks,
21        "网裂区域数": total_regions,
22        "裂缝总长度": total_length_m,
23        "网裂总面积": total_area_m2,
24        "平均宽度": avg_width,
25        "较重及严重数量": severe_count,
26        "病害指数": risk_index,
27        "类型分布": dict(type_counter),
28        "等级分布": dict(level_counter),
29    }
30    from __future__ import annotations
31    import hashlib
32    import json
33    from pathlib import Path
34    from typing import List, Optional
35    from .models import User, AnnotationTask
36    class StorageManager:
37        """本地 JSON 存储。演示系统不依赖数据库，便于直接运行。"""
38        def __init__(self, data_dir: Path):
39            self.data_dir = Path(data_dir)
40            self.data_dir.mkdir(parents=True, exist_ok=True)
41            self.users_path = self.data_dir / "users.json"
42            self.tasks_path = self.data_dir / "tasks.json"
43            self._ensure_files()
44        def _ensure_files(self):
45            if not self.users_path.exists():
46                admin = User(username="admin", password_hash=self.hash_passw
47                self.users_path.write_text(json.dumps([admin.to_dict()], ens
48            if not self.tasks_path.exists():
49                self.tasks_path.write_text("[]", encoding="utf-8")
50    @staticmethod
51

```

```

1 def hash_password(password: str) -> str:
2     return hashlib.sha256((password + "road_crack_salt").encode("utf
3 def load_users(self) -> List[User]:
4     try:
5         data = json.loads(self.users_path.read_text(encoding="utf-8"
6         return [User.from_dict(x) for x in data]
7     except Exception:
8         return []
9 def save_users(self, users: List[User]):
10     self.users_path.write_text(json.dumps([u.to_dict() for u in user
11 def register_user(self, username: str, password: str, role: str = "标
12     username = username.strip()
13     if len(username) < 2:
14         return False, "用户名至少需要 2 个字符"
15     if len(password) < 6:
16         return False, "密码至少需要 6 位"
17     users = self.load_users()
18     if any(u.username == username for u in users):
19         return False, "用户名已存在"
20     users.append(User(username=username, password_hash=self.hash_pas
21     self.save_users(users)
22     return True, "注册成功"
23 def verify_user(self, username: str, password: str) -> Optional[User
24     hashed = self.hash_password(password)
25     for user in self.load_users():
26         if user.username == username and user.password_hash == hashe
27         return user
28     return None
29 def load_tasks(self) -> List[AnnotationTask]:
30     try:
31         data = json.loads(self.tasks_path.read_text(encoding="utf-8"
32         return [AnnotationTask.from_dict(x) for x in data]
33     except Exception:
34         return []
35 def save_tasks(self, tasks: List[AnnotationTask]):
36     self.tasks_path.write_text(json.dumps([t.to_dict() for t in task
37 def upsert_task(self, task: AnnotationTask):
38     tasks = self.load_tasks()
39     found = False
40     for i, old in enumerate(tasks):
41         if old.task_id == task.task_id:
42             task.touch()
43             tasks[i] = task
44             found = True
45             break
46     if not found:
47         tasks.append(task)
48     self.save_tasks(tasks)
49 from __future__ import annotations
50 from .geometry import distance

```

```

1  from .models import WidthSample
2  class CrackWidthGauge:
3      """宽度卡尺: 通过裂缝两侧两个点计算宽度。"""
4      @staticmethod
5      def make_sample(p1, p2, meter_per_pixel):
6          pixel_width = distance(p1, p2)
7          if meter_per_pixel:
8              width_mm = pixel_width * meter_per_pixel * 1000
9          else:
10             width_mm = pixel_width
11             return WidthSample(p1=p1, p2=p2, width_mm=width_mm)
12     @staticmethod
13     def status_text(width_mm: float) -> str:
14         if width_mm <= 0:
15             return "未采样"
16         if width_mm < 2:
17             return "细微裂缝"
18         if width_mm < 5:
19             return "明显裂缝"
20         if width_mm < 8:
21             return "较宽裂缝"
22         return "宽度超限, 建议复核"
23 from __future__ import annotations
24 from PyQt6.QtCore import QObject, pyqtSignal
25 from typing import Optional, List
26 from app.core.models import AnnotationTask, CrackObject
27 from app.core.snap import CrackSnapAssistant
28 class AppState(QObject):
29     task_changed = pyqtSignal()
30     selection_changed = pyqtSignal()
31     mode_changed = pyqtSignal(str)
32     message = pyqtSignal(str)
33     view_flags_changed = pyqtSignal()
34     def __init__(self):
35         super().__init__()
36         self.tasks: List[AnnotationTask] = []
37         self.current_task: Optional[AnnotationTask] = None
38         self.selected_crack_id: Optional[str] = None
39         self.mode: str = '浏览'
40         self.snap_assistant = CrackSnapAssistant(False, 10)
41         self.temporary_points = []
42         self.width_pending_point = None
43         self.region_pending_points = []
44         self.show_scale_ruler = True
45         self.show_reference_grid = True
46         self.reference_grid_opacity = 38 # 0-100, 画在影像上的网格透明度
47         self.overlay_line_opacity = 100
48         self.street_view_live = False
49     def set_tasks(self, tasks: List[AnnotationTask]):
50         self.tasks = tasks

```



```

1         if tasks and self.current_task is None:
2             self.current_task = tasks[0]
3         self.task_changed.emit()
4     def set_current_task(self, task: Optional[AnnotationTask]):
5         self.current_task = task
6         self.selected_crack_id = None
7         self.temporary_points.clear()
8         self.width_pending_point = None
9         self.region_pending_points.clear()
10        self.task_changed.emit()
11        self.selection_changed.emit()
12    def set_mode(self, mode: str):
13        self.mode = mode
14        self.temporary_points.clear()
15        self.width_pending_point = None
16        self.region_pending_points.clear()
17        self.mode_changed.emit(mode)
18        self.message.emit(f'当前模式: {mode}')
19    def set_show_scale_ruler(self, flag: bool):
20        self.show_scale_ruler = bool(flag)
21        self.view_flags_changed.emit()
22    def set_show_reference_grid(self, flag: bool):
23        self.show_reference_grid = bool(flag)
24        self.view_flags_changed.emit()
25    def set_reference_grid_opacity(self, value: int):
26        self.reference_grid_opacity = max(0, min(100, int(value)))
27        self.view_flags_changed.emit()
28    def set_overlay_line_opacity(self, value: int):
29        self.overlay_line_opacity = max(15, min(100, int(value)))
30        self.view_flags_changed.emit()
31    def selected_crack(self) -> Optional[CrackObject]:
32        if not self.current_task or not self.selected_crack_id:
33            return None
34        for crack in self.current_task.cracks:
35            if crack.crack_id == self.selected_crack_id:
36                return crack
37        return None
38    def select_crack(self, crack_id: Optional[str]):
39        self.selected_crack_id = crack_id
40        self.selection_changed.emit()
41        if crack_id:
42            self.message.emit(f'已选中 {crack_id}')
43    def add_task(self, task: AnnotationTask):
44        self.tasks.append(task)
45        self.set_current_task(task)
46        self.message.emit(f'已创建任务: {task.road_name}')
47    from __future__ import annotations
48    from PyQt6.QtWidgets import QWidget, QMessageBox, QInputDialog
49    from PyQt6.QtGui import QPainter, QPixmap, QPen, QColor, QBrush, QFont,
50    from PyQt6.QtCore import Qt, QPointF, pyqtSignal, QRectF

```

```

1  from app.core.models import CrackObject, CrackRegion
2  from app.core.geometry import polyline_length_real, smooth_polyline, nea
3  from app.core.calibration import ImageCalibrationEngine
4  from app.core.width_gauge import CrackWidthGauge
5  from app.core.region_analyzer import CrackRegionAnalyzer
6  from app.core.severity import CrackSeverityEvaluator
7  class CrackAnnotationCanvas(QWidget):
8      canvas_changed = pyqtSignal()
9      request_save = pyqtSignal()
10     def __init__(self, state, parent=None):
11         super().__init__(parent)
12         self.state = state
13         self.setMinimumSize(780, 540)
14         self.setMouseTracking(True)
15         self.scale = 0.75
16         self.offset = QPointF(30, 30)
17         self.panning = False
18         self.last_mouse = QPointF(0, 0)
19         self.dragging_node = None
20         self.hover_world = None
21         self.pixmap = QPixmap()
22         self.qimage = QImage()
23         self.calibration_points = []
24         self.state.task_changed.connect(self.reload_task_image)
25         self.state.selection_changed.connect(self.update)
26         self.state.mode_changed.connect(lambda _: self.update())
27         self.state.view_flags_changed.connect(self.update)
28     def reload_task_image(self):
29         task = self.state.current_task
30         if task and task.image_path:
31             self.pixmap = QPixmap(task.image_path)
32             self.qimage = self.pixmap.toImage() if not self.pixmap.isNull()
33         else:
34             self.pixmap = QPixmap()
35             self.qimage = QImage()
36         self.fit_image(); self.update()
37     def fit_image(self):
38         if self.pixmap.isNull():
39             self.scale = 1.0; self.offset = QPointF(20, 20); return
40         sw = max(1, self.width() - 80); sh = max(1, self.height() - 80)
41         self.scale = min(sw / self.pixmap.width(), sh / self.pixmap.height())
42         self.offset = QPointF((self.width() - self.pixmap.width() * self.scale) / 2,
43                               (self.height() - self.pixmap.height() * self.scale) / 2)
44     def world_to_screen(self, p):
45         return QPointF(p[0] * self.scale + self.offset.x(), p[1] * self.scale + self.offset.y())
46     def screen_to_world(self, pos):
47         return ((pos.x() - self.offset.x()) / self.scale, (pos.y() - self.offset.y()) / self.scale)
48     def paintEvent(self, event):
49         painter = QPainter(self)
50         painter.setRenderHint(QPainter.RenderHint.Antialiasing)
51         painter.fillRect(self.rect(), QColor('#eef2f7'))

```

```

1         if self.pixmap.isNull():
2             painter.setPen(QColor('#7f93ab'))
3             painter.setFont(QFont('Microsoft YaHei', 18, QFont.Weight.Bold))
4             painter.drawText(self.rect(), Qt.AlignmentFlag.AlignCenter,
5                             return
6         shadow_rect = QRectF(self.offset.x() + 6, self.offset.y() + 6, s
7         painter.fillRect(shadow_rect, QColor(150, 163, 184, 55))
8         painter.drawPixmap(int(self.offset.x()), int(self.offset.y()), i
9         if self.state.show_reference_grid:
10             self._draw_image_grid(painter)
11             painter.setPen(QPen(QColor('#c9d3df'), 1))
12             painter.drawRect(QRectF(self.offset.x(), self.offset.y(), self.p
13             self._draw_overlays(painter)
14             self._draw_hud(painter)
15             if self.state.show_scale_ruler:
16                 self._draw_scale_ruler(painter)
17     def _draw_image_grid(self, painter):
18         if self.pixmap.isNull():
19             return
20         alpha = int(255 * max(0, min(100, self.state.reference_grid_opac
21         if alpha <= 0:
22             return
23         left = self.offset.x()
24         top = self.offset.y()
25         right = left + self.pixmap.width() * self.scale
26         bottom = top + self.pixmap.height() * self.scale
27         step = max(30, int(200 * self.scale))
28         painter.save()
29         painter.setClipRect(QRectF(left, top, right - left, bottom - top
30         painter.setPen(QPen(QColor(46, 125, 255, alpha), 1, Qt.PenStyle.
31         x = left
32         while x <= right:
33             painter.drawLine(int(x), int(top), int(x), int(bottom))
34             x += step
35         y = top
36         while y <= bottom:
37             painter.drawLine(int(left), int(y), int(right), int(y))
38             y += step
39         painter.restore()
40     def _severity_color(self, severity, selected=False):
41         if selected:
42             return QColor('#1677ff')
43         opacity = int(255 * max(15, min(100, self.state.overlay_line_opa
44         color = {
45             '轻微': QColor('#31b057'),
46             '一般': QColor('#f1aa1f'),
47             '较重': QColor('#f07c32'),
48             '严重': QColor('#df4b63'),
49             '未评估': QColor('#7b8da1'),
50         }.get(severity, QColor('#7b8da1'))

```

```

1     color.setAlpha(opacity)
2     return color
3 def _draw_overlays(self, painter):
4     task = self.state.current_task
5     if not task:
6         return
7     for candidate in task.candidates:
8         if candidate.status != '待确认' or len(candidate.points) < 2:
9             continue
10        painter.setPen(QPen(QColor('#8c5bff'), 2, Qt.PenStyle.DashLi
11        pts = [self.world_to_screen(p) for p in candidate.points]
12        for a, b in zip(pts[:-1], pts[1:]):
13            painter.drawLine(a, b)
14            center = pts[len(pts)//2]
15            painter.setPen(QColor('#6d46cb'))
16            painter.drawText(center + QPointF(5, -5), f'疑似 {candidate.sc
17    for region in task.regions:
18        if len(region.polygon_points) < 3:
19            continue
20        poly = QPolygonF([self.world_to_screen(p) for p in region.po
21        painter.setBrush(QBrush(QColor(255, 196, 64, 55)))
22        painter.setPen(QPen(self._severity_color(region.severity), 2
23        painter.drawPolygon(poly)
24        c = poly.boundingRect().center()
25        painter.setPen(QColor('#7d5b00'))
26        painter.drawText(c, f'{region.region_id}\n{region.area_m2:.2
27    for crack in task.cracks:
28        if len(crack.points) < 2:
29            continue
30        selected = crack.crack_id == self.state.selected_crack_id
31        color = self._severity_color(crack.severity, selected)
32        width = 5 if selected else 3
33        if crack.locked:
34            pen = QPen(color, width, Qt.PenStyle.SolidLine)
35        elif crack.review_status == '已退回':
36            pen = QPen(QColor('#df4b63'), width, Qt.PenStyle.DashDot
37        else:
38            pen = QPen(color, width, Qt.PenStyle.SolidLine)
39        pen.setCapStyle(Qt.PenCapStyle.RoundCap); pen.setJoinStyle(Q
40        painter.setPen(pen)
41        pts = [self.world_to_screen(p) for p in crack.points]
42        for a, b in zip(pts[:-1], pts[1:]):
43            painter.drawLine(a, b)
44        painter.setBrush(QBrush(QColor('white')))
45        for pt in pts:
46            painter.setPen(QPen(color, 2))
47            painter.drawEllipse(pt, 4, 4)
48        mid = pts[len(pts)//2]
49        if selected:
50            painter.setPen(QPen(QColor('ffffff'), 1)); painter.setB
51

```

```

1         painter.drawRoundedRect(int(mid.x()+8), int(mid.y()-28),
2         painter.setPen(QColor('white'))
3         painter.drawText(int(mid.x()+14), int(mid.y()-11), crack
4         painter.setPen(QPen(QColor('#4fb8d9'), 2))
5         for sample in crack.width_samples:
6             a, b = self.world_to_screen(sample.p1), self.world_t
7             painter.drawLine(a, b); painter.drawEllipse(a, 3, 3)
8             mid2 = QPointF((a.x()+b.x())/2, (a.y()+b.y())/2)
9             painter.drawText(mid2 + QPointF(4, -4), f'{sample.wi
10     if self.state.temporary_points:
11         painter.setPen(QPen(QColor('#1677ff'), 2, Qt.PenStyle.DashLi
12         pts = [self.world_to_screen(p) for p in self.state.temporary
13         for a, b in zip(pts[:-1], pts[1:]):
14             painter.drawLine(a, b)
15         painter.setBrush(QBrush(QColor('#1677ff')))
16         for pt in pts:
17             painter.drawEllipse(pt, 4, 4)
18     if self.state.region_pending_points:
19         painter.setPen(QPen(QColor('#ffb703'), 2, Qt.PenStyle.DashLi
20         pts = [self.world_to_screen(p) for p in self.state.region_pe
21         for a, b in zip(pts[:-1], pts[1:]):
22             painter.drawLine(a, b)
23         for pt in pts:
24             painter.drawEllipse(pt, 4, 4)
25     if self.calibration_points:
26         painter.setPen(QPen(QColor('#0f172a'), 2))
27         pts = [self.world_to_screen(p) for p in self.calibration_poi
28         for p in pts: painter.drawEllipse(p, 6, 6)
29         if len(pts) == 2: painter.drawLine(pts[0], pts[1])
30 def _draw_hud(self, painter):
31     task = self.state.current_task
32     text = [f'模式: {self.state.mode}']
33     if task:
34         text.append(f'任务: {task.road_name}')
35         text.append(ImageCalibrationEngine.describe(task.meter_per_p
36         text.append(f'裂缝: {len(task.cracks)} 网裂区: {len(task.regions)}')
37     painter.setBrush(QColor(255, 255, 255, 232)); painter.setPen(QCo
38     painter.drawRoundedRect(14, 14, 390, 96, 12, 12)
39     painter.setPen(QColor('#24364b'))
40     y = 36
41     for line in text:
42         painter.drawText(28, y, line); y += 21
43 def _draw_scale_ruler(self, painter):
44     task = self.state.current_task
45     if self.pixmap.isNull():
46         return
47     img_x = self.offset.x()
48     img_y = self.offset.y()
49     img_w = self.pixmap.width() * self.scale
50     img_h = self.pixmap.height() * self.scale

```

```

1 ruler_h = 26
2 ruler_w = 34
3 top_y = max(0, img_y - ruler_h)
4 left_x = max(0, img_x - ruler_w)
5 painter.setPen(QPen(QColor('#c8d3e1'), 1))
6 painter.setBrush(QBrush(QColor(255, 255, 255, 238)))
7 painter.drawRect(QRectF(img_x, top_y, img_w, ruler_h))
8 painter.drawRect(QRectF(left_x, img_y, ruler_w, img_h))
9 painter.fillRect(QRectF(left_x, top_y, ruler_w, ruler_h), QColor
10 meter_per_pixel = task.meter_per_pixel if task else None
11 if meter_per_pixel:
12     major_px = max(1, 0.5 / meter_per_pixel) # 每 0.5 米一个大刻度
13     minor_px = major_px / 5
14     unit_label = '米'
15 else:
16     major_px = 200
17     minor_px = 50
18     unit_label = '像素'
19 painter.setFont(QFont('Microsoft YaHei', 8))
20 painter.setPen(QPen(QColor('#31445b'), 1))
21 def label_for(v_px):
22     if meter_per_pixel:
23         return f'{v_px * meter_per_pixel:.1f}'
24     return f'{int(v_px)}'
25 max_px_x = self.pixmap.width()
26 tick = 0.0
27 while tick <= max_px_x:
28     sx = img_x + tick * self.scale
29     if sx >= img_x - 1 and sx <= img_x + img_w + 1:
30         is_major = abs((tick / major_px) - round(tick / major_px
31         h = 14 if is_major else 7
32         painter.drawLine(int(sx), int(top_y + ruler_h), int(sx),
33         if is_major:
34             painter.drawText(int(sx + 3), int(top_y + 11), label
35     tick += minor_px
36 max_px_y = self.pixmap.height()
37 tick = 0.0
38 while tick <= max_px_y:
39     sy = img_y + tick * self.scale
40     if sy >= img_y - 1 and sy <= img_y + img_h + 1:
41         is_major = abs((tick / major_px) - round(tick / major_px
42         w = 18 if is_major else 9
43         painter.drawLine(int(left_x + ruler_w), int(sy), int(lef
44         if is_major:
45             painter.save()
46             painter.translate(int(left_x + 8), int(sy - 3))
47             painter.rotate(-90)
48             painter.drawText(0, 0, label_for(tick))
49             painter.restore()
50     tick += minor_px
51

```

```

1 painter.setPen(QColor('#31445b'))
2 painter.drawText(int(left_x + 4), int(top_y + 18), unit_label)
3 if self.hover_world and self._inside_image(self.hover_world):
4     hx = img_x + self.hover_world[0] * self.scale
5     hy = img_y + self.hover_world[1] * self.scale
6     painter.setPen(QPen(QColor('#1677ff'), 2))
7     painter.drawLine(int(hx), int(top_y), int(hx), int(top_y + r
8     painter.drawLine(int(left_x), int(hy), int(left_x + ruler_w)
9 def wheelEvent(self, event):
10     old_scale = self.scale; delta = event.angleDelta().y(); factor =
11     self.scale = max(0.08, min(6.0, self.scale * factor))
12     pos = event.position(); wx = (pos.x() - self.offset.x()) / old_s
13     self.offset = QPointF(pos.x() - wx * self.scale, pos.y() - wy *
14     self.update()
15 def mousePressEvent(self, event):
16     if self.pixmap.isNull():
17         return
18     if event.button() == Qt.MouseButton.MiddleButton:
19         self.panning = True; self.last_mouse = event.position(); ret
20     if event.button() == Qt.MouseButton.RightButton:
21         self.finish_current_shape(); return
22     if event.button() != Qt.MouseButton.LeftButton:
23         return
24     world = self.screen_to_world(event.position()); mode = self.stat
25     if mode == '浏览':
26         if self.try_start_drag(world):
27             return
28         self.pick_crack_or_candidate(world)
29     elif mode == '折线裂缝':
30         self.add_polyline_point(world)
31     elif mode == '比例标定':
32         self.add_calibration_point(world)
33     elif mode == '宽度卡尺':
34         self.add_width_point(world)
35     elif mode == '网裂圈选':
36         self.state.region_pending_points.append(world)
37         self.state.message.emit('已添加网裂区域顶点, 右键闭合区域')
38     self.update()
39 def mouseMoveEvent(self, event):
40     if self.panning:
41         d = event.position() - self.last_mouse; self.offset += d; se
42     world = self.screen_to_world(event.position()); self.hover_world
43     if self.dragging_node is not None:
44         crack, idx = self.dragging_node
45         if not crack.locked and self._inside_image(world):
46             crack.points[idx] = world
47             CrackSeverityEvaluator.refresh_crack(crack, self.state.c
48             self.canvas_changed.emit(); self.update()
49 def mouseReleaseEvent(self, event):
50     if event.button() == Qt.MouseButton.MiddleButton:

```

```

1         self.panning = False
2         if event.button() == Qt.MouseButton.LeftButton and self.dragging
3             self.dragging_node = None
4             self.refresh_all_measurements(); self.request_save.emit(); s
5     def mouseDoubleClickEvent(self, event):
6         if self.state.mode == '折线裂缝':
7             self.finish_polyline()
8     def try_start_drag(self, world):
9         task = self.state.current_task
10        if not task:
11            return False
12        radius = 14 / max(self.scale, 0.1)
13        ordered = []
14        selected = self.state.selected_crack()
15        if selected:
16            ordered.append(selected)
17        ordered.extend([c for c in task.cracks if c is not selected])
18        for crack in ordered:
19            if crack.locked:
20                continue
21            idx = nearest_point_index(crack.points, world, radius=radius)
22            if idx >= 0:
23                self.state.select_crack(crack.crack_id)
24                self.dragging_node = (crack, idx)
25                self.state.message.emit('正在拖动裂缝节点，可用于修正自动识别结果')
26            return True
27        return False
28    def add_polyline_point(self, world):
29        task = self.state.current_task
30        if not task:
31            QMessageBox.warning(self, '没有任务', '请先在影像任务舱创建任务'); return
32        if not self._inside_image(world):
33            return
34        x, y = self.state.snap_assistant.snap_qimage(self.qimage, *world
35        self.state.temporary_points.append((x, y))
36        self.state.message.emit(self.state.snap_assistant.last_message i
37    def finish_polyline(self):
38        task = self.state.current_task
39        if not task or len(self.state.temporary_points) < 2:
40            self.state.temporary_points.clear(); return
41        crack = CrackObject(points=list(self.state.temporary_points))
42        CrackSeverityEvaluator.refresh_crack(crack, task.meter_per_pixel)
43        task.cracks.append(crack); task.status = '标注中'; task.touch(); se
44        self.canvas_changed.emit(); self.request_save.emit(); self.state
45    def add_calibration_point(self, world):
46        if not self._inside_image(world): return
47        self.calibration_points.append(world)
48        if len(self.calibration_points) == 2:
49            real, ok = QInputDialog.getDouble(self, '比例尺标定', '请输入两点之间实际距
50            if ok:

```



```

1         try:
2             mpp = ImageCalibrationEngine.calibrate_by_two_points
3             task = self.state.current_task; task.meter_per_pixel
4             self.canvas_changed.emit(); self.request_save.emit()
5         except Exception as e:
6             QMessageBox.warning(self, '标定失败', str(e))
7             self.calibration_points.clear()
8             if self.state.mode == '比例标定':
9                 self.state.set_mode('浏览')
10    def add_width_point(self, world):
11        crack = self.state.selected_crack(); task = self.state.current_t
12        if not task: return
13        if not crack:
14            QMessageBox.information(self, '先选择裂缝', '请先在画布或右侧卡片中选择一条裂缝');
15        if crack.locked:
16            QMessageBox.warning(self, '裂缝已锁定', '复核通过并锁定的裂缝不能继续添加宽度采样');
17        if self.state.width_pending_point is None:
18            self.state.width_pending_point = world; self.state.message.e
19        else:
20            sample = CrackWidthGauge.make_sample(self.state.width_pendin
21            crack.width_samples.append(sample); self.state.width_pending
22            self.state.message.emit(f'宽度采样完成: {sample.width_mm:.2f}mm, {Cr
23            self.canvas_changed.emit(); self.request_save.emit()
24    def finish_region(self):
25        task = self.state.current_task; pts = list(self.state.region_pen
26        if not task or len(pts) < 3:
27            self.state.region_pending_points.clear(); return
28        region = CrackRegion(polygon_points=pts)
29        region.area_m2 = CrackRegionAnalyzer.area(pts, task.meter_per_pi
30        total_len = sum(polyline_length_real(c.points, task.meter_per_pi
31        if not task.meter_per_pixel: total_len = total_len / 180
32        region.density = CrackRegionAnalyzer.density(total_len, region.a
33        task.regions.append(region); task.status = '标注中'; self.state.reg
34        self.canvas_changed.emit(); self.request_save.emit()
35    def finish_current_shape(self):
36        if self.state.mode == '折线裂缝': self.finish_polyline()
37        elif self.state.mode == '网裂圈选': self.finish_region()
38        else: self.pick_crack_or_candidate(self.screen_to_world(self.map
39        self.update()
40    def pick_crack_or_candidate(self, world):
41        task = self.state.current_task
42        if not task: return
43        for candidate in task.candidates:
44            if candidate.status != '待确认': continue
45            if any(distance(world, p) < 15 / max(self.scale, 0.1) for p
46                reply = QMessageBox.question(self, '疑似裂缝', '是否将该疑似暗线转为正式
47                if reply == QMessageBox.StandardButton.Yes:
48                    crack = CrackObject(points=list(candidate.points), s
49                    CrackSeverityEvaluator.refresh_crack(crack, task.met
50                    task.cracks.append(crack); candidate.status = '已转正';

```

```

1         self.canvas_changed.emit(); self.request_save.emit()
2     else:
3         candidate.status = '已忽略'; self.state.message.emit('疑
4         return
5     best_id = None
6     best_d = 24 / max(self.scale, 0.1)
7     for crack in task.cracks:
8         if len(crack.points) < 2:
9             continue
10        for a, b in zip(crack.points[:-1], crack.points[1:]):
11            d = self._point_segment_distance(world, a, b)
12            if d < best_d:
13                best_id = crack.crack_id
14                best_d = d
15    self.state.select_crack(best_id)
16 def undo_last_point(self):
17     if self.state.mode == '折线裂缝' and self.state.temporary_points:
18         self.state.temporary_points.pop()
19     elif self.state.mode == '网裂圈选' and self.state.region_pending_poi
20         self.state.region_pending_points.pop()
21     self.update()
22 def smooth_selected_crack(self):
23     crack = self.state.selected_crack()
24     if not crack: return False
25     if crack.locked:
26         QMessageBox.warning(self, '已锁定', '该裂缝已锁定, 不能平滑。'); return Fal
27     crack.points = smooth_polyline(crack.points, 0.55, 2); CrackSeve
28 def delete_selected_any_annotation(self):
29     task = self.state.current_task
30     crack = self.state.selected_crack()
31     if not task or not crack:
32         return False, '请先选择一条标注线'
33     if crack.locked:
34         return False, '该裂缝已锁定, 请先在复核纠偏舱解除锁定'
35     task.cracks = [c for c in task.cracks if c.crack_id != crack.cra
36     task.touch()
37     self.state.select_crack(None)
38     self.canvas_changed.emit()
39     self.request_save.emit()
40     self.update()
41     return True, '已删除选中标注线'
42 def delete_auto_detected_annotations(self):
43     task = self.state.current_task
44     if not task:
45         return 0
46     before = len(task.cracks)
47     task.cracks = [c for c in task.cracks if c.source != '自动识别']
48     removed = before - len(task.cracks)
49     if removed:
50         task.touch()

```

```

1         self.state.select_crack(None)
2         self.canvas_changed.emit()
3         self.request_save.emit()
4         self.update()
5     return removed
6 def clear_candidates(self):
7     task = self.state.current_task
8     if not task:
9         return 0
10    count = len(task.candidates)
11    task.candidates.clear()
12    task.touch()
13    self.canvas_changed.emit()
14    self.request_save.emit()
15    self.update()
16    return count
17 def delete_selected_crack(self):
18     task = self.state.current_task; crack = self.state.selected_crack
19     if not task or not crack: return False
20     if crack.locked:
21         QMessageBox.warning(self, '已锁定', '已锁定裂缝不能删除。'); return False
22     task.cracks = [c for c in task.cracks if c.crack_id != crack.crack_id]
23 def refresh_all_measurements(self):
24     task = self.state.current_task
25     if not task: return
26     for crack in task.cracks:
27         for sample in crack.width_samples:
28             sample.width_mm = CrackWidthGauge.make_sample(sample.p1,
29             CrackSeverityEvaluator.refresh_crack(crack, task.meter_per_pixel))
30     for region in task.regions:
31         region.area_m2 = CrackRegionAnalyzer.area(region.polygon_points)
32         total_len = sum(polyline_length_real(c.points, task.meter_per_pixel) for c in region.polygons)
33         if not task.meter_per_pixel: total_len = total_len / 180
34         region.density = CrackRegionAnalyzer.density(total_len, region.area_m2)
35     task.touch(); self.update()
36 def _point_segment_distance(self, p, a, b):
37     px, py = p
38     ax, ay = a
39     bx, by = b
40     vx, vy = bx - ax, by - ay
41     wx, wy = px - ax, py - ay
42     denom = vx * vx + vy * vy
43     if denom <= 1e-9:
44         return distance(p, a)
45     t = max(0.0, min(1.0, (wx * vx + wy * vy) / denom))
46     q = (ax + t * vx, ay + t * vy)
47     return distance(p, q)
48 def _inside_image(self, p):
49     if self.pixmap.isNull(): return False
50     return 0 <= p[0] <= self.pixmap.width() and 0 <= p[1] <= self.pixmap.height()

```

```

1  from __future__ import annotations
2  from PyQt6.QtWidgets import QDialog, QVBoxLayout, QHBoxLayout, QLabel, Q
3  from PyQt6.QtCore import Qt
4  from app.core.storage import StorageManager
5  from app.core.models import User
6  class LoginDialog(QDialog):
7      def __init__(self, storage: StorageManager):
8          super().__init__()
9          self.storage = storage
10         self.current_user: User | None = None
11         self.setWindowTitle("路面裂缝人工标注与测量辅助软件 - 登录")
12         self.resize(440, 360)
13         layout = QVBoxLayout(self)
14         layout.setContentsMargins(30, 24, 30, 24)
15         layout.setSpacing(14)
16         title = QLabel("路面裂缝人工标注与测量辅助软件")
17         title.setObjectName("TitleLabel")
18         title.setAlignment(Qt.AlignmentFlag.AlignCenter)
19         subtitle = QLabel("裂缝标注 · 卡尺测量 · 复核纠偏 · 修复估算")
20         subtitle.setAlignment(Qt.AlignmentFlag.AlignCenter)
21         layout.addWidget(title)
22         layout.addWidget(subtitle)
23         self.username = QLineEdit("admin")
24         self.username.setPlaceholderText("用户名")
25         self.password = QLineEdit("admin123")
26         self.password.setPlaceholderText("密码")
27         self.password.setEchoMode(QLineEdit.EchoMode.Password)
28         self.role = QComboBox()
29         self.role.addItems(["标注员", "复核员", "管理员"])
30         layout.addWidget(QLabel("用户名"))
31         layout.addWidget(self.username)
32         layout.addWidget(QLabel("密码"))
33         layout.addWidget(self.password)
34         layout.addWidget(QLabel("注册角色"))
35         layout.addWidget(self.role)
36         row = QHBoxLayout()
37         login_btn = QPushButton("登录")
38         login_btn.setObjectName("PrimaryButton")
39         reg_btn = QPushButton("注册新账号")
40         row.addWidget(login_btn)
41         row.addWidget(reg_btn)
42         layout.addLayout(row)
43         tips = QLabel("默认账号: admin / admin123。注册后可直接登录。")
44         tips.setWordWrap(True)
45         tips.setAlignment(Qt.AlignmentFlag.AlignCenter)
46         layout.addWidget(tips)
47         login_btn.clicked.connect(self.login)
48         reg_btn.clicked.connect(self.register)
49     def login(self):
50         user = self.storage.verify_user(self.username.text(), self.passw

```

```

1         if not user:
2             QMessageBox.warning(self, "登录失败", "用户名或密码不正确")
3             return
4         self.current_user = user
5         self.accept()
6     def register(self):
7         ok, msg = self.storage.register_user(self.username.text(), self.
8         if ok:
9             QMessageBox.information(self, "注册成功", msg + ", 现在可以登录。")
10        else:
11            QMessageBox.warning(self, "注册失败", msg)
12    from __future__ import annotations
13    from pathlib import Path
14    from PyQt6.QtWidgets import (
15        QMainWindow, QWidget, QVBoxLayout, QHBoxLayout, QPushButton, QLabel,
16        QStackedWidget, QListWidget, QListWidgetItem, QSplitter, QFrame, QPr
17    )
18    from PyQt6.QtCore import Qt, QTimer
19    from app.ui.app_state import AppState
20    from app.ui.canvas import CrackAnnotationCanvas
21    from app.ui.widgets import title_label
22    from app.core.quality import AnnotationQualityChecker
23    from app.core.geometry import polyline_length_real
24    from app.ui.panels.image_task_panel import ImageTaskPanel
25    from app.ui.panels.calibration_panel import CalibrationPanel
26    from app.ui.panels.drawing_panel import DrawingPanel
27    from app.ui.panels.snap_panel import SnapPanel
28    from app.ui.panels.width_panel import WidthPanel
29    from app.ui.panels.region_panel import RegionPanel
30    from app.ui.panels.diagnosis_panel import DiagnosisPanel
31    from app.ui.panels.missing_panel import MissingPanel
32    from app.ui.panels.street_view_panel import StreetViewPanel
33    from app.ui.panels.review_panel import ReviewPanel
34    from app.ui.panels.repair_panel import RepairPanel
35    from app.ui.panels.statistics_panel import StatisticsPanel
36    from app.ui.panels.report_panel import ReportPanel
37    class MainWindow(QMainWindow):
38        PANEL_CLASSES = [
39            ('影像任务舱', ImageTaskPanel),
40            ('比例标定舱', CalibrationPanel),
41            ('裂缝描绘舱', DrawingPanel),
42            ('磁吸辅助舱', SnapPanel),
43            ('宽度卡尺舱', WidthPanel),
44            ('网裂圈选舱', RegionPanel),
45            ('病害诊断舱', DiagnosisPanel),
46            ('自动标注舱', MissingPanel),
47            ('街景标注舱', StreetViewPanel),
48            ('复核纠偏舱', ReviewPanel),
49            ('修复估算舱', RepairPanel),
50            ('路段统计舱', StatisticsPanel),

```

```

1      ('成果报告舱', ReportPanel),
2  ]
3  def __init__(self, storage, user, project_root: Path):
4      super().__init__()
5      self.storage = storage
6      self.user = user
7      self.project_root = Path(project_root)
8      self.state = AppState()
9      self.state.set_tasks(self.storage.load_tasks())
10     self.setWindowTitle('路面裂缝人工标注与测量辅助软件')
11     self.resize(1560, 930)
12     self._build_ui()
13     self._connect_signals()
14     self.refresh_all()
15     self.select_panel(0)
16     if not self.state.tasks:
17         QTimer.singleShot(500, lambda: self.statusBar().showMessage(
18 def _build_ui(self):
19     central = QWidget(); self.setCentralWidget(central)
20     root_layout = QVBoxLayout(central)
21     root_layout.setContentsMargins(12, 10, 12, 10)
22     root_layout.setSpacing(8)
23     title_row = QHBoxLayout()
24     title_row.addWidget(title_label('路面裂缝人工标注与测量辅助软件'))
25     subtitle = QLabel('白色清爽风 | 类 PS 工作台 | 支持自动识别 + 人工修正')
26     subtitle.setStyleSheet('color:#6a7d95; font-size:12px;')
27     title_row.addWidget(subtitle)
28     title_row.addStretch(1)
29     self.user_label = QLabel(f'当前用户: {self.user.username} | 角色: {self.us
30     self.user_label.setAlignment(Qt.AlignmentFlag.AlignRight | Qt.Al
31     title_row.addWidget(self.user_label)
32     root_layout.addLayout(title_row)
33     quick_row = QHBoxLayout()
34     for text, mode in [('浏览', '浏览'), ('描绘裂缝', '折线裂缝'), ('宽度卡尺', '宽度卡
35         btn = QPushButton(text)
36         btn.clicked.connect(lambda checked=False, m=mode: self.state
37         quick_row.addWidget(btn)
38     quick_row.addStretch(1)
39     del_btn = QPushButton('删除选中标注线')
40     del_btn.clicked.connect(self.delete_selected_annotation)
41     quick_row.addWidget(del_btn)
42     save_btn = QPushButton('保存全部任务'); save_btn.setObjectName('Primar
43     quick_row.addWidget(save_btn)
44     root_layout.addLayout(quick_row)
45     splitter = QSplitter(Qt.Orientation.Horizontal)
46     root_layout.addWidget(splitter, 1)
47     left = self._build_left_menu(); splitter.addWidget(left)
48     center = QWidget(); center_layout = QVBoxLayout(center); center_
49     self.canvas = CrackAnnotationCanvas(self.state)
50     self.track = QListWidget(); self.track.setMaximumHeight(120); se

```

```

1 center_layout.addWidget(self.canvas, 1)
2 center_layout.addWidget(QLabel('量测轨道条: 点击裂缝块可定位并选中, 浏览模式下可拖动节点修正。'))
3 center_layout.addWidget(self.track)
4 splitter.addWidget(center)
5 right = QWidget(); right_layout = QVBoxLayout(right); right_layo
6 self.quality_bar = QProgressBar(); self.quality_bar.setRange(0,
7 right_layout.addWidget(QLabel('标注质量评分'))
8 right_layout.addWidget(self.quality_bar)
9 self.stack = QStackedWidget(); self.panels = []
10 for _, cls in self.PANEL_CLASSES:
11     panel = cls(self.state, self.storage, self.project_root, sel
12     self.panels.append(panel)
13     scroll = QScrollArea()
14     scroll.setWidgetResizable(True)
15     scroll.setFrameShape(QFrame.Shape.NoFrame)
16     scroll.setHorizontalScrollBarPolicy(Qt.ScrollBarPolicy.Scr
17     scroll.setWidget(panel)
18     self.stack.addWidget(scroll)
19 right_layout.addWidget(self.stack, 1)
20 self.crack_cards_title = QLabel('裂缝诊断卡片流 (紧凑)')
21 self.crack_cards_title.setStyleSheet('color:#223042; font-weight
22 right_layout.addWidget(self.crack_cards_title)
23 self.crack_cards = QListWidget()
24 self.crack_cards.setMinimumHeight(64)
25 self.crack_cards.setMaximumHeight(118)
26 self.crack_cards.setAlternatingRowColors(True)
27 self.crack_cards.itemClicked.connect(self.card_clicked)
28 right_layout.addWidget(self.crack_cards, 0)
29 splitter.addWidget(right)
30 splitter.setSizes([185, 900, 500])
31 self.statusBar().showMessage('系统已启动')
32 def _build_left_menu(self):
33     frame = QFrame(); frame.setObjectName('PanelCard')
34     layout = QVBoxLayout(frame); layout.setContentsMargins(10, 12, 1
35     layout.addWidget(QLabel('功能工作舱'))
36     self.menu_buttons = []
37     for idx, (name, _) in enumerate(self.PANEL_CLASSES):
38         btn = QPushButton(name)
39         btn.clicked.connect(lambda checked=False, i=idx: self.select
40         self.menu_buttons.append(btn); layout.addWidget(btn)
41     layout.addStretch(1)
42     return frame
43 def _connect_signals(self):
44     self.state.task_changed.connect(self.refresh_all)
45     self.state.selection_changed.connect(self.refresh_all)
46     self.state.message.connect(lambda msg: self.statusBar().showMess
47     self.state.view_flags_changed.connect(self.canvas.update)
48     self.canvas.canvas_changed.connect(self.refresh_all)
49     self.canvas.request_save.connect(self.save_all)
50 def select_panel(self, index: int):

```

```

1     edit_pages = {1, 2, 4, 5}
2     if index not in edit_pages and self.state.mode in ('折线裂缝', '网裂圈选')
3         if self.state.mode == '折线裂缝' and len(self.state.temporary_po
4             self.canvas.finish_polyline()
5         elif self.state.mode == '网裂圈选' and len(self.state.region_pen
6             self.canvas.finish_region()
7             self.state.set_mode('浏览')
8     if index == 6 and self.state.current_task and not self.state.sel
9         self.state.select_crack(self.state.current_task.cracks[0].cr
10    self.stack.setCurrentIndex(index)
11    for i, btn in enumerate(self.menu_buttons):
12        btn.setObjectName('PrimaryButton' if i == index else '')
13        btn.style().unpolish(btn); btn.style().polish(btn)
14    self.statusBar().showMessage(f'已进入: {self.PANEL_CLASSES[index][0]}')
15    def refresh_all(self):
16        self.refresh_crack_cards(); self.refresh_track(); self.refresh_q
17    def refresh_quality(self):
18        task = self.state.current_task
19        score, details = AnnotationQualityChecker.score(task) if task el
20        self.quality_bar.setValue(score); self.quality_bar.setFormat(f'{
21        self.quality_bar.setToolTip(' '.join(details[:6]) if score < 85
22    def refresh_crack_cards(self):
23        self.crack_cards.clear(); task = self.state.current_task
24        if not task:
25            self.crack_cards.addItem('暂无任务: 上方功能区可先导入影像或打开样例')
26            return
27        if not task.cracks:
28            self.crack_cards.addItem('暂无裂缝: 可在自动标注舱识别, 或在裂缝描绘舱手工标注')
29            return
30        for crack in task.cracks:
31            length = polyline_length_real(crack.points, task.meter_per_p
32            if not task.meter_per_pixel:
33                length = length / 180
34            text = (
35                f'{crack.crack_id} | {crack.source} | {crack.crack_type} | {cr
36                f'长{length:.2f}m | 均宽{crack.avg_width_mm:.2f}mm | 最大{crack.m
37            )
38            item = QListWidgetItem(text); item.setData(Qt.ItemDataRole.U
39            self.crack_cards.addItem(item)
40            if crack.crack_id == self.state.selected_crack_id:
41                self.crack_cards.setCurrentItem(item)
42    def refresh_track(self):
43        self.track.clear(); task = self.state.current_task
44        if not task: return
45        for crack in task.cracks:
46            length = polyline_length_real(crack.points, task.meter_per_p
47            if not task.meter_per_pixel:
48                length = length / 180
49            block = '█' * max(2, min(28, int(length * 3) + 2))
50            text = f'{block} {crack.crack_id} | {crack.severity} | {length:

```



```

1         item = QListWidgetItem(text); item.setData(Qt.ItemDataRole.U
2         self.track.addItem(item)
3     def card_clicked(self, item):
4         self.state.select_crack(item.data(Qt.ItemDataRole.UserRole)); se
5     def track_clicked(self, item):
6         self.state.select_crack(item.data(Qt.ItemDataRole.UserRole)); se
7     def delete_selected_annotation(self):
8         ok, msg = self.canvas.delete_selected_any_annotation()
9         self.statusBar().showMessage(msg, 5000)
10        self.refresh_all()
11    def save_all(self):
12        self.storage.save_tasks(self.state.tasks)
13        self.statusBar().showMessage('全部任务已保存', 4000)
14    def closeEvent(self, event):
15        self.save_all(); event.accept()
16    from __future__ import annotations
17    from PyQt6.QtWidgets import QWidget, QVBoxLayout, QLabel, QPushButton, Q
18    from PyQt6.QtCore import Qt
19    from app.ui.widgets import PanelCard, primary_button, InfoBox
20    from app.core.calibration import ImageCalibrationEngine
21    class CalibrationPanel(QWidget):
22        def __init__(self, state, storage, project_root, canvas=None, parent
23        super().__init__(parent)
24        self.state = state
25        self.storage = storage
26        self.canvas = canvas
27        layout = QVBoxLayout(self)
28        card = PanelCard('比例标定舱')
29        card.layout.addWidget(QLabel('两点标定: 点击按钮后在画布上点两个标尺端点, 再输入实际距离。'))
30        btn = primary_button('进入两点标定模式')
31        btn.clicked.connect(lambda: self.state.set_mode('比例标定'))
32        card.layout.addWidget(btn)
33        close_calib_btn = QPushButton('关闭两点标定模式')
34        close_calib_btn.clicked.connect(self.close_calibration_mode)
35        card.layout.addWidget(close_calib_btn)
36        self.default_mpp = QDoubleSpinBox()
37        self.default_mpp.setRange(0.01, 50)
38        self.default_mpp.setValue(0.5)
39        self.default_mpp.setSuffix(' 毫米/像素')
40        card.layout.addWidget(QLabel('固定相机默认比例'))
41        card.layout.addWidget(self.default_mpp)
42        default_btn = QPushButton('应用默认比例并重算')
43        default_btn.clicked.connect(self.apply_default)
44        card.layout.addWidget(default_btn)
45        self.ruler_check = QCheckBox('显示图像边缘比例尺')
46        self.ruler_check.setChecked(self.state.show_scale_ruler)
47        self.ruler_check.toggled.connect(self.state.set_show_scale_ruler)
48        card.layout.addWidget(self.ruler_check)
49        self.grid_check = QCheckBox('在图像上显示参考网格')
50        self.grid_check.setChecked(self.state.show_reference_grid)

```

```

1      self.grid_check.toggled.connect(self.state.set_show_reference_gr
2      card.layout.addWidget(self.grid_check)
3      card.layout.addWidget(QLabel('参考网格透明度'))
4      self.grid_opacity = QSlider(Qt.Orientation.Horizontal)
5      self.grid_opacity.setRange(0, 100)
6      self.grid_opacity.setValue(self.state.reference_grid_opacity)
7      self.grid_opacity.valueChanged.connect(self.state.set_reference_
8      card.layout.addWidget(self.grid_opacity)
9      card.layout.addWidget(QLabel('标注线透明度'))
10     self.line_opacity = QSlider(Qt.Orientation.Horizontal)
11     self.line_opacity.setRange(15, 100)
12     self.line_opacity.setValue(self.state.overlay_line_opacity)
13     self.line_opacity.valueChanged.connect(self.state.set_overlay_li
14     card.layout.addWidget(self.line_opacity)
15     recalc_btn = QPushButton('重新计算全部裂缝量测值')
16     recalc_btn.clicked.connect(self.recalculate)
17     card.layout.addWidget(recalc_btn)
18     delete_btn = QPushButton('删除选中标注线')
19     delete_btn.clicked.connect(self.delete_selected)
20     card.layout.addWidget(delete_btn)
21     self.info = InfoBox()
22     card.layout.addWidget(self.info)
23     layout.addWidget(card)
24     self.state.task_changed.connect(self.refresh)
25     self.state.view_flags_changed.connect(self.refresh)
26     self.refresh()
27 def close_calibration_mode(self):
28     self.state.set_mode('浏览')
29     if self.canvas:
30         self.canvas.calibration_points.clear()
31         self.canvas.update()
32     QMessageBox.information(self, '两点标定', '已关闭两点标定模式，画布已回到浏览模式。')
33 def apply_default(self):
34     task = self.state.current_task
35     if not task:
36         return
37     task.meter_per_pixel = self.default_mpp.value() / 1000
38     if self.canvas:
39         self.canvas.refresh_all_measurements()
40         self.canvas.canvas_changed.emit()
41         self.canvas.update()
42     self.storage.upsert_task(task)
43     self.refresh()
44     QMessageBox.information(self, '比例已应用', '全部测量值已按默认比例重新计算')
45 def recalculate(self):
46     if self.canvas:
47         self.canvas.refresh_all_measurements()
48         self.canvas.canvas_changed.emit()
49         self.canvas.update()
50     if self.state.current_task:

```

```

1         self.storage.upsert_task(self.state.current_task)
2         self.refresh()
3         QMessageBox.information(self, '已重算', '长度、宽度、面积和等级已刷新')
4     def delete_selected(self):
5         if not self.canvas:
6             return
7         ok, msg = self.canvas.delete_selected_any_annotation()
8         QMessageBox.information(self, '删除标注线', msg if ok else msg)
9     def refresh(self):
10        task = self.state.current_task
11        text = [ImageCalibrationEngine.describe(task.meter_per_pixel) if
12        text.append(f'图像边缘比例尺: {"显示" if self.state.show_scale_ruler else
13        text.append(f'图像参考网格: {"显示" if self.state.show_reference_grid els
14        text.append(f'网格透明度: {self.state.reference_grid_opacity}%')
15        text.append(f'标注线透明度: {self.state.overlay_line_opacity}%')
16        text.append('参考网格会覆盖在图像上, 不再只显示在画布背景中。')
17        self.info.setText('\n'.join(text))
18    from __future__ import annotations
19    from PyQt6.QtWidgets import QWidget, QVBoxLayout, QLabel, QPushButton, Q
20    from app.ui.widgets import PanelCard, primary_button, danger_button, Inf
21    from app.core.severity import CrackSeverityEvaluator
22    from app.core.geometry import polyline_length_real
23    from app.core.region_analyzer import CrackRegionAnalyzer
24    class DiagnosisPanel(QWidget):
25        def __init__(self, state, storage, project_root, canvas=None, parent
26            super().__init__(parent)
27            self.state = state
28            self.storage = storage
29            self.canvas = canvas
30            layout = QVBoxLayout(self)
31            card = PanelCard('病害诊断舱')
32            self.type_box = QComboBox()
33            self.type_box.addItems(['横向裂缝', '纵向裂缝', '斜向裂缝', '网状裂缝', '龟裂', '块
34            card.layout.addWidget(QLabel('人工确认裂缝类型'))
35            card.layout.addWidget(self.type_box)
36            apply = primary_button('采纳/修改类型并重新评估等级')
37            apply.clicked.connect(self.apply_type)
38            card.layout.addWidget(apply)
39            first = QPushButton('自动选择第一条裂缝')
40            first.clicked.connect(self.select_first_crack)
41            card.layout.addWidget(first)
42            explain = QPushButton('显示等级判定依据')
43            explain.clicked.connect(self.explain)
44            card.layout.addWidget(explain)
45            batch = QPushButton('批量诊断裂缝和网裂区域')
46            batch.clicked.connect(self.batch)
47            card.layout.addWidget(batch)
48            delete_line = danger_button('删除选中标注线')
49            delete_line.clicked.connect(self.delete_selected_line)
50            card.layout.addWidget(delete_line)

```

```

1     self.summary_list = QListWidget()
2     card.layout.addWidget(QLabel('诊断对象汇总'))
3     card.layout.addWidget(self.summary_list)
4     self.info = InfoBox()
5     card.layout.addWidget(self.info)
6     layout.addWidget(card)
7     self.state.selection_changed.connect(self.refresh)
8     self.state.task_changed.connect(self.refresh)
9     if self.canvas:
10         self.canvas.canvas_changed.connect(self.refresh)
11     self.refresh()
12 def select_first_crack(self):
13     task = self.state.current_task
14     if task and task.cracks:
15         self.state.select_crack(task.cracks[0].crack_id)
16         QMessageBox.information(self, '已选择', f'已选中 {task.cracks[0].c
17     else:
18         QMessageBox.information(self, '暂无裂缝', '当前任务还没有折线裂缝，可先自动标注或人工
19 def apply_type(self):
20     task = self.state.current_task
21     c = self.state.selected_crack()
22     if task and not c and task.cracks:
23         c = task.cracks[0]
24         self.state.select_crack(c.crack_id)
25     if not c or not task:
26         QMessageBox.information(self, '无法诊断', '当前没有可诊断的折线裂缝。网裂区域可使用“
27         return
28     c.crack_type = self.type_box.currentText()
29     reasons = CrackSeverityEvaluator.refresh_crack(c, task.meter_per
30     task.touch()
31     self.storage.upsert_task(task)
32     if self.canvas:
33         self.canvas.canvas_changed.emit()
34         self.canvas.update()
35     self.info.setText('\n'.join(reasons))
36     self.refresh()
37     QMessageBox.information(self, '诊断完成', f'{c.crack_id} 已评估为: {c.sev
38 def explain(self):
39     c = self.state.selected_crack()
40     task = self.state.current_task
41     if not c or not task:
42         QMessageBox.information(self, '未选择裂缝', '请先点击画布裂缝线、右侧卡片或底部轨道选
43         return
44     reasons = CrackSeverityEvaluator.refresh_crack(c, task.meter_per
45     length = polyline_length_real(c.points, task.meter_per_pixel)
46     length = length if task.meter_per_pixel else length / 180
47     QMessageBox.information(
48         self,
49         '判定依据',
50         f'长度: {length:.2f}米\n 平均宽度: {c.avg_width_mm:.2f}mm\n 最大宽度: {c.max

```

```

1     )
2     def batch(self):
3         task = self.state.current_task
4         if not task:
5             return
6         crack_count = 0
7         for c in task.cracks:
8             CrackSeverityEvaluator.refresh_crack(c, task.meter_per_pixel
9             crack_count += 1
10        total_len = sum(polyline_length_real(c.points, task.meter_per_pi
11        if not task.meter_per_pixel:
12            total_len = total_len / 180
13        region_count = 0
14        for r in task.regions:
15            r.area_m2 = CrackRegionAnalyzer.area(r.polygon_points, task.
16            r.density = CrackRegionAnalyzer.density(total_len, r.area_m2
17            r.severity = CrackRegionAnalyzer.severity(r.area_m2, r.densi
18            region_count += 1
19        task.touch()
20        self.storage.upsert_task(task)
21        if self.canvas:
22            self.canvas.canvas_changed.emit()
23            self.canvas.update()
24        self.refresh()
25        QMessageBox.information(self, '批量完成', f'已诊断裂缝 {crack_count} 条, 网裂
26    def delete_selected_line(self):
27        if not self.canvas:
28            return
29        ok, msg = self.canvas.delete_selected_any_annotation()
30        self.refresh()
31        QMessageBox.information(self, '删除标注线', msg)
32    def refresh(self):
33        self.summary_list.clear()
34        task = self.state.current_task
35        if not task:
36            self.info.setText('尚未选择任务。')
37            return
38        for c in task.cracks:
39            self.summary_list.addItem(f'裂缝 | {c.crack_id} | {c.crack_type} | {
40        for r in task.regions:
41            self.summary_list.addItem(f'网裂区 | {r.region_id} | 面积{r.area_m2:.
42        c = self.state.selected_crack()
43        if c:
44            self.type_box.setCurrentText(c.crack_type if c.crack_type e1
45            length = polyline_length_real(c.points, task.meter_per_pixel
46            length = length if task.meter_per_pixel else length / 180
47            self.info.setText(
48                f'当前选中: {c.crack_id}\n'
49                f'系统建议: {c.suggestion_type}\n'
50                f'当前类型: {c.crack_type}\n'

```

```

1         f'长度: {length:.2f}米\n'
2         f'平均宽度: {c.avg_width_mm:.2f}mm\n'
3         f'严重等级: {c.severity}'
4     )
5     elif task.cracks:
6         self.info.setText('当前任务已有裂缝, 但未选中。可点击“自动选择第一条裂缝”, 或点击画布线段/右侧卡片
7     elif task.regions:
8         self.info.setText('当前任务已有网裂区域。网裂区域不需要选择折线, 可点击“批量诊断裂缝和网裂区域”。'
9     else:
10        pending = len(self.state.region_pending_points)
11        if pending >= 3:
12            self.info.setText('检测到有未闭合的网裂圈选点。进入本舱时系统会尽量自动闭合, 若未生成请回到网
13        else:
14            self.info.setText('未发现已完成标注。请先进入自动标注舱或裂缝描绘舱生成裂缝对象。')
15
16 from __future__ import annotations
17 from PyQt6.QtWidgets import QWidget, QVBoxLayout, QPushButton, QMessageBox
18 from app.ui.widgets import PanelCard, primary_button, danger_button, Inf
19 class DrawingPanel(QWidget):
20     def __init__(self, state, storage, project_root, canvas=None, parent
21     super().__init__(parent)
22     self.state = state
23     self.storage = storage
24     self.canvas = canvas
25     layout = QVBoxLayout(self)
26     card = PanelCard('裂缝描绘舱')
27     btn = primary_button('进入折线裂缝描绘模式')
28     btn.clicked.connect(lambda: self.state.set_mode('折线裂缝'))
29     card.layout.addWidget(btn)
30     finish = QPushButton('结束当前裂缝')
31     finish.clicked.connect(lambda: self.canvas.finish_polyline() if
32     card.layout.addWidget(finish)
33     undo = QPushButton('撤销上一个节点')
34     undo.clicked.connect(lambda: self.canvas.undo_last_point() if se
35     card.layout.addWidget(undo)
36     smooth = QPushButton('平滑选中裂缝')
37     smooth.clicked.connect(self.smooth)
38     card.layout.addWidget(smooth)
39     delete = danger_button('删除选中裂缝')
40     delete.clicked.connect(self.delete)
41     card.layout.addWidget(delete)
42     browse = QPushButton('返回浏览选择模式')
43     browse.clicked.connect(lambda: self.state.set_mode('浏览'))
44     card.layout.addWidget(browse)
45     self.info = InfoBox('左键连续点选裂缝中心线, 右键或双击结束。浏览模式下可拖动选中裂缝的节点, 便于人工调整自
46     card.layout.addWidget(self.info)
47     layout.addWidget(card)
48     self.state.selection_changed.connect(self.refresh)
49     self.refresh()
50     def smooth(self):
51         ok = self.canvas.smooth_selected_crack() if self.canvas else Fal

```

```

1     QMessageBox.information(self, '线条平滑', '已对裂缝节点进行平滑处理' if ok else
2 def delete(self):
3     ok = self.canvas.delete_selected_crack() if self.canvas else False
4     QMessageBox.information(self, '删除结果', '已删除选中裂缝' if ok else '没有可删
5 def refresh(self):
6     c = self.state.selected_crack()
7     if c:
8         self.info.setText(
9             f'选中裂缝: {c.crack_id}\n'
10            f'来源: {c.source}\n'
11            f'节点数: {len(c.points)}\n'
12            f'类型: {c.crack_type}\n'
13            f'等级: {c.severity}\n'
14            f'复核: {c.review_status}\n'
15            f'提示: 在浏览模式下可直接拖动节点。'
16        )
17 from __future__ import annotations
18 from pathlib import Path
19 from PyQt6.QtWidgets import (
20     QWidget, QVBoxLayout, QLabel, QPushButton, QFileDialog, QLineEdit,
21     QComboBox, QListWidget, QMessageBox
22 )
23 from app.core.models import AnnotationTask
24 from app.ui.widgets import PanelCard, primary_button, InfoBox
25 class ImageTaskPanel(QWidget):
26     def __init__(self, state, storage, project_root, canvas=None, parent
27         super().__init__(parent)
28         self.state = state
29         self.storage = storage
30         self.project_root = Path(project_root)
31         self.canvas = canvas
32         layout = QVBoxLayout(self)
33         card = PanelCard('影像任务舱')
34         self.road = QLineEdit('G320 示范路段')
35         self.station_start = QLineEdit('K12+000')
36         self.station_end = QLineEdit('K12+100')
37         self.location_name = QLineEdit('武汉市东西湖区示范路段')
38         self.longitude = QLineEdit('114.305678')
39         self.latitude = QLineEdit('30.572123')
40         self.direction = QComboBox(); self.direction.addItem('上行', '下行
41         self.road_level = QComboBox(); self.road_level.addItem('城市主干路'
42         self.section_note = QLineEdit('常规路面巡检影像')
43         self.pavement = QComboBox(); self.pavement.addItem('沥青路面', '水泥
44         self.lane = QComboBox(); self.lane.addItem('上行一车道', '上行二车道', '
45         for label, widget in [
46             ('道路名称', self.road),
47             ('起点桩号', self.station_start),
48             ('终点桩号', self.station_end),
49             ('位置说明', self.location_name),
50             ('经度', self.longitude),

```

```

1         ('纬度', self.latitude),
2         ('道路方向', self.direction),
3         ('道路等级', self.road_level),
4         ('路面类型', self.pavement),
5         ('车道', self.lane),
6         ('路段备注', self.section_note),
7     ]:
8         card.layout.addWidget(QLabel(label))
9         card.layout.addWidget(widget)
10    import_btn = primary_button('导入本地路面影像并创建任务')
11    sample_btn = QPushButton('打开更真实的内置样例影像')
12    save_meta = QPushButton('更新当前任务路段信息')
13    refresh_btn = QPushButton('刷新任务列表')
14    delete_line = QPushButton('删除选中标注线')
15    import_btn.clicked.connect(self.import_image)
16    sample_btn.clicked.connect(self.open_sample)
17    save_meta.clicked.connect(self.update_current_metadata)
18    refresh_btn.clicked.connect(self.refresh_list)
19    delete_line.clicked.connect(self.delete_selected_line)
20    card.layout.addWidget(import_btn)
21    card.layout.addWidget(sample_btn)
22    card.layout.addWidget(save_meta)
23    card.layout.addWidget(delete_line)
24    card.layout.addWidget(refresh_btn)
25    self.task_list = QListWidget()
26    self.task_list.setMaximumHeight(150)
27    self.task_list.itemClicked.connect(self.select_task)
28    card.layout.addWidget(QLabel('任务卡片'))
29    card.layout.addWidget(self.task_list)
30    self.info = InfoBox('导入影像后会生成标注任务；每个路段可填写位置、坐标、方向、等级和备注。')
31    card.layout.addWidget(self.info)
32    layout.addWidget(card)
33    self.state.task_changed.connect(self.refresh_list)
34    self.refresh_list()
35    def import_image(self):
36        path, _ = QFileDialog.getOpenFileName(self, '选择路面影像', str(self.p
37        if path:
38            self.create_task(path)
39    def open_sample(self):
40        samples = sorted((self.project_root / 'assets' / 'samples').glob
41        if not samples:
42            QMessageBox.warning(self, '缺少样例', '未找到样例影像')
43            return
44        row = max(0, len(self.state.tasks) % len(samples))
45        self.create_task(str(samples[row]))
46    def create_task(self, image_path):
47        task = AnnotationTask(
48            road_name=self.road.text().strip() or '未命名道路',
49            image_path=str(image_path),
50            pavement_type=self.pavement.currentText(),
51

```



```

1         station_start=self.station_start.text().strip() or 'K0+000',
2         station_end=self.station_end.text().strip() or 'K0+100',
3         lane=self.lane.currentText(),
4         location_name=self.location_name.text().strip(),
5         longitude=self.longitude.text().strip(),
6         latitude=self.latitude.text().strip(),
7         road_level=self.road_level.currentText(),
8         direction=self.direction.currentText(),
9         section_note=self.section_note.text().strip(),
10        source_type='普通路面影像',
11        annotator='当前用户',
12        status='待标注',
13    )
14    self.state.add_task(task)
15    self.storage.upsert_task(task)
16    self.info.setText(
17        f'已创建任务: {task.road_name}\n'
18        f'位置: {task.location_name}\n'
19        f'坐标: {task.latitude}, {task.longitude}\n'
20        f'影像: {Path(task.image_path).name}\n'
21        '建议流程: 比例标定 → 自动识别/人工描绘 → 宽度估算 → 复核。'
22    )
23    def update_current_metadata(self):
24        task = self.state.current_task
25        if not task:
26            return
27        task.road_name = self.road.text().strip() or task.road_name
28        task.station_start = self.station_start.text().strip() or task.s
29        task.station_end = self.station_end.text().strip() or task.stati
30        task.location_name = self.location_name.text().strip()
31        task.longitude = self.longitude.text().strip()
32        task.latitude = self.latitude.text().strip()
33        task.direction = self.direction.currentText()
34        task.road_level = self.road_level.currentText()
35        task.pavement_type = self.pavement.currentText()
36        task.lane = self.lane.currentText()
37        task.section_note = self.section_note.text().strip()
38        task.touch()
39        self.storage.upsert_task(task)
40        self.refresh_list()
41        QMessageBox.information(self, '路段信息已更新', '当前任务的位置、坐标、方向和备注已保存。')
42    def delete_selected_line(self):
43        if not self.canvas:
44            return
45        ok, msg = self.canvas.delete_selected_any_annotation()
46        self.refresh_list()
47        QMessageBox.information(self, '删除标注线', msg)
48    def refresh_list(self):
49        self.task_list.clear()
50        for task in self.state.tasks:

```

```

1         self.task_list.addItem(
2             f'{task.road_name} | {task.station_start}-{task.station_en
3         )
4     def select_task(self, item):
5         row = self.task_list.currentRow()
6         if 0 <= row < len(self.state.tasks):
7             task = self.state.tasks[row]
8             self.state.set_current_task(task)
9             self.road.setText(task.road_name)
10            self.station_start.setText(task.station_start)
11            self.station_end.setText(task.station_end)
12            self.location_name.setText(task.location_name)
13            self.longitude.setText(task.longitude)
14            self.latitude.setText(task.latitude)
15            self.direction.setCurrentText(task.direction)
16            self.road_level.setCurrentText(task.road_level)
17            self.pavement.setCurrentText(task.pavement_type)
18            self.lane.setCurrentText(task.lane)
19            self.section_note.setText(task.section_note)
20            self.info.setText(
21                f'当前任务: {task.road_name}\n'
22                f'位置: {task.location_name}\n'
23                f'坐标: {task.latitude}, {task.longitude}\n'
24                f'影像: {Path(task.image_path).name}\n'
25                '模式建议: 先标定比例尺, 再自动识别或描绘裂缝。'
26            )
27 from __future__ import annotations
28 from PyQt6.QtWidgets import QWidget, QVBoxLayout, QLabel, QPushButton, Q
29 from app.ui.widgets import PanelCard, primary_button, danger_button, Inf
30 from app.core.missing_scanner import MissingCrackScanner
31 from app.core.auto_detect import CrackAutoDetector
32 class MissingPanel(QWidget):
33     def __init__(self, state, storage, project_root, canvas=None, parent
34         super().__init__(parent)
35         self.state = state
36         self.storage = storage
37         self.canvas = canvas
38         layout = QVBoxLayout(self)
39         card = PanelCard('自动标注舱')
40         card.layout.addWidget(QLabel('识别策略'))
41         self.sensitivity = QComboBox()
42         self.sensitivity.addItem('保守')
43         self.sensitivity.addItem('标准')
44         self.sensitivity.addItem('增强')
45         self.sensitivity.setCurrentText('标准')
46         card.layout.addWidget(self.sensitivity)
47         auto_cover = primary_button('自动识别当前影像裂缝(覆盖旧识别)')
48         auto_cover.clicked.connect(lambda: self.auto_detect_cracks(overw
49         card.layout.addWidget(auto_cover)
50         auto_append = QPushButton('追加自动识别结果')
51         auto_append.clicked.connect(lambda: self.auto_detect_cracks(over
52         card.layout.addWidget(auto_append)

```

```

1 scan = QPushButton('仅扫描疑似漏标暗线')
2 scan.clicked.connect(self.scan)
3 card.layout.addWidget(scan)
4 delete_selected = danger_button('删除选中标注线')
5 delete_selected.clicked.connect(self.delete_selected_line)
6 card.layout.addWidget(delete_selected)
7 delete_auto = QPushButton('删除全部自动识别标注线')
8 delete_auto.clicked.connect(self.delete_auto_lines)
9 card.layout.addWidget(delete_auto)
10 ignore = QPushButton('忽略全部待确认候选')
11 ignore.clicked.connect(self.ignore_all)
12 card.layout.addWidget(ignore)
13 clear_candidates = QPushButton('清空疑似候选线')
14 clear_candidates.clicked.connect(self.clear_candidates)
15 card.layout.addWidget(clear_candidates)
16 self.list = QListWidget()
17 self.list.setMaximumHeight(210)
18 self.list.itemClicked.connect(self.select_from_list)
19 card.layout.addWidget(QLabel('自动识别 / 候选列表'))
20 card.layout.addWidget(self.list)
21 self.info = InfoBox(
22     '自动标注已改为局部暗线增强算法。默认“标准”策略会兼顾识别率和误检控制；'
23     '如果图像裂缝很浅，再改用“标准”或“增强”。'
24 )
25 card.layout.addWidget(self.info)
26 layout.addWidget(card)
27 self.canvas.canvas_changed.connect(self.refresh) if self.canvas
28 self.state.task_changed.connect(self.refresh)
29 self.state.selection_changed.connect(self.refresh)
30 self.refresh()
31 def auto_detect_cracks(self, overwrite: bool = True):
32     task = self.state.current_task
33     if not task:
34         return
35     if overwrite:
36         task.cracks = [c for c in task.cracks if c.source != '自动识别']
37     strategy = self.sensitivity.currentText()
38     cracks = CrackAutoDetector.detect_from_image(
39         task.image_path,
40         task.meter_per_pixel,
41         sensitivity=strategy,
42     )
43     if not cracks:
44         QMessageBox.warning(
45             self,
46             '识别结果',
47             f'当前“{strategy}”策略未识别到明显裂缝。\\n'
48             '可尝试切换到“标准/增强”，或使用人工描绘模式。'
49         )
50     self.refresh()
51

```

```

1         return
2     task.cracks.extend(cracks)
3     task.status = f'自动识别后待复核 ({strategy}) '
4     task.touch()
5     self.storage.upsert_task(task)
6     if cracks:
7         self.state.select_crack(cracks[0].crack_id)
8     if self.canvas:
9         self.canvas.canvas_changed.emit()
10        self.canvas.update()
11    self.refresh()
12    mode_text = '覆盖旧识别后' if overwrite else '追加'
13    QMessageBox.information(
14        self,
15        '识别完成',
16        f'{mode_text}生成 {len(cracks)} 条裂缝，识别策略: {strategy}。\\n'
17        '可在浏览模式选中裂缝节点并拖动，继续人工调整。'
18    )
19    def scan(self):
20        task = self.state.current_task
21        if not task or not self.canvas:
22            return
23        cands = MissingCrackScanner.scan_qimage(self.canvas.qimage, task)
24        task.candidates.extend(cands)
25        task.touch()
26        self.storage.upsert_task(task)
27        self.canvas.canvas_changed.emit()
28        self.canvas.update()
29        self.refresh()
30        QMessageBox.information(self, '扫描完成', f'发现 {len(cands)} 条疑似暗线候选')
31    def delete_selected_line(self):
32        if not self.canvas:
33            return
34        ok, msg = self.canvas.delete_selected_any_annotation()
35        self.refresh()
36        QMessageBox.information(self, '删除标注线', msg)
37    def delete_auto_lines(self):
38        if not self.canvas:
39            return
40        count = self.canvas.delete_auto_detected_annotations()
41        self.refresh()
42        QMessageBox.information(self, '删除自动识别线', f'已删除 {count} 条自动识别标注线')
43    def ignore_all(self):
44        task = self.state.current_task
45        if not task:
46            return
47        for c in task.candidates:
48            if c.status == '待确认':
49                c.status = '已忽略'
50        task.touch()

```

```

1         self.storage.upsert_task(task)
2         if self.canvas:
3             self.canvas.canvas_changed.emit()
4             self.canvas.update()
5         self.refresh()
6     def clear_candidates(self):
7         if not self.canvas:
8             return
9         count = self.canvas.clear_candidates()
10        self.refresh()
11        QMessageBox.information(self, '清空候选线', f'已清空 {count} 条疑似候选线')
12    def select_from_list(self, item):
13        data = item.data(256)
14        if isinstance(data, str) and data.startswith('裂缝-'):
15            self.state.select_crack(data)
16    def refresh(self):
17        self.list.clear()
18        task = self.state.current_task
19        if not task:
20            self.info.setText('尚未选择任务。请先在影像任务舱或街景标注舱打开图片。')
21            return
22        for c in task.cracks[-40:]:
23            self.list.addItem(f'裂缝 | {c.crack_id} | {c.source} | {c.severity} |
24            self.list.item(self.list.count() - 1).setData(256, c.crack_i
25        for c in task.candidates:
26            self.list.addItem(f'候选 | {c.candidate_id} | 得分{c.score:.0f} | {c.s
27        self.info.setText(
28            f'当前任务裂缝数: {len(task.cracks)}\n'
29            f'自动识别裂缝: {len([x for x in task.cracks if x.source == "自动识别"]
30            f'待确认候选: {len([x for x in task.candidates if x.status == "待确认
31            f'当前策略: {self.sensitivity.currentText()}\n'
32            '建议: 默认用“标准”; 如果误检仍多, 切换“保守”; 漏检明显时再用“增强”。'
33        )
34    from __future__ import annotations
35    from PyQt6.QtWidgets import QWidget, QVBoxLayout, QLabel, QPushButton, Q
36    from app.ui.widgets import PanelCard, primary_button, danger_button, Inf
37    class RegionPanel(QWidget):
38        def __init__(self, state, storage, project_root, canvas=None, parent
39            super().__init__(parent); self.state=state; self.storage=storage
40            layout=QVBoxLayout(self); card=PanelCard("网裂圈选舱")
41            btn=primary_button("进入网裂区域圈选模式"); btn.clicked.connect(lambda: se
42            finish=QPushButton("闭合并生成网裂区域"); finish.clicked.connect(lambda:
43            undo=QPushButton("撤销区域顶点"); undo.clicked.connect(lambda: self.ca
44            clear=QPushButton("清空当前任务所有网裂区"); clear.clicked.connect(self.cle
45            delete_line=danger_button("删除选中标注线"); delete_line.clicked.conne
46            self.list=QListWidget(); card.layout.addWidget(QLabel("网裂区域列表"))
47            self.info=InfoBox("左键圈选网裂或龟裂区域, 右键闭合, 也可点击“闭合并生成网裂区域”。切换到诊断、复核、统计等
48            card.layout.addWidget(self.info); layout.addWidget(card)
49            self.state.task_changed.connect(self.refresh); self.canvas.canva
50    def delete_selected_line(self):

```

```

1         if not self.canvas:
2             return
3         ok, msg = self.canvas.delete_selected_any_annotation()
4         self.refresh()
5         QMessageBox.information(self, "删除标注线", msg)
6     def clear_regions(self):
7         task=self.state.current_task
8         if not task: return
9         task.regions.clear(); self.storage.upsert_task(task); self.canva
10    def refresh(self):
11        self.list.clear(); task=self.state.current_task
12        if not task: return
13        for r in task.regions: self.list.addItem(f"{r.region_id} | 面积{r.ar
14    from __future__ import annotations
15    from PyQt6.QtWidgets import QWidget, QVBoxLayout, QLabel, QPushButton, Q
16    from app.ui.widgets import PanelCard, primary_button, InfoBox
17    from app.core.repair import RepairMaterialEstimator
18    from app.core.geometry import polyline_length_real
19    class RepairPanel(QWidget):
20        def __init__(self, state, storage, project_root, canvas=None, parent
21            super().__init__(parent); self.state=state; self.storage=storage
22            layout=QVBoxLayout(self); card=PanelCard("修复估算舱")
23            self.method=QComboBox(); self.method.addItem(RepairMaterialEsti
24            card.layout.addWidget(QLabel("修复方式")); card.layout.addWidget(sel
25            rec=QPushButton("按等级自动推荐方式"); rec.clicked.connect(self.recommend
26            est=primary_button("估算选中裂缝材料用量"); est.clicked.connect(self.estim
27            batch=QPushButton("批量估算当前任务全部裂缝"); batch.clicked.connect(self.ba
28            self.list=QListWidget(); card.layout.addWidget(QLabel("维修材料清单"))
29            self.info=InfoBox("标注结果可直接转为材料估算: 灌缝胶按体积估算, 罩面与铣刨按面积/厚度估算。")
30            card.layout.addWidget(self.info); layout.addWidget(card)
31            self.state.selection_changed.connect(self.refresh); self.canvas.
32    def recommend(self):
33        c=self.state.selected_crack()
34        if not c: return
35        self.method.setCurrentText(RepairMaterialEstimator.recommend(c.s
36    def estimate(self):
37        c=self.state.selected_crack(); task=self.state.current_task
38        if not c or not task: return
39        length=polyline_length_real(c.points, task.meter_per_pixel); len
40        method,amount,unit=RepairMaterialEstimator.estimate_for_crack(le
41        c.repair_method=method; c.material_amount=amount; c.material_uni
42        self.storage.upsert_task(task); self.refresh(); QMessageBox.info
43    def batch(self):
44        task=self.state.current_task
45        if not task: return
46        for c in task.cracks:
47            length=polyline_length_real(c.points, task.meter_per_pixel);
48            method,amount,unit=RepairMaterialEstimator.estimate_for_crac
49            c.repair_method=method; c.material_amount=amount; c.material
50        self.storage.upsert_task(task); self.refresh(); QMessageBox.info

```

```

1     def refresh(self):
2         self.list.clear(); task=self.state.current_task
3         if not task: return
4         total={}
5         for c in task.cracks:
6             self.list.addItem(f"{c.crack_id} | {c.repair_method} | {c.materi
7             total[(c.repair_method,c.material_unit)] = total.get((c.repa
8             summary='\n'.join([f'{k[0]}: {v:.2f}{k[1]}' for k,v in total.item
9             self.info.setText('材料汇总: \n'+summary)
10 from __future__ import annotations
11 from pathlib import Path
12 from PyQt6.QtWidgets import QWidget, QVBoxLayout, QLabel, QPushButton, Q
13 from app.ui.widgets import PanelCard, primary_button, InfoBox
14 from app.core.report import CrackReportGenerator
15 from app.core.quality import AnnotationQualityChecker
16 class ReportPanel(QWidget):
17     def __init__(self, state, storage, project_root, canvas=None, parent
18         super().__init__(parent); self.state=state; self.storage=storage
19         self.generator=CrackReportGenerator(self.project_root/'exports')
20         layout=QVBoxLayout(self); card=PanelCard('成果报告舱')
21         check=QPushButton('导出前完整性检查'); check.clicked.connect(self.check_
22         json_btn=QPushButton('导出 JSON 标注数据'); json_btn.clicked.connect(sel
23         csv_btn=QPushButton('导出 CSV 测量表'); csv_btn.clicked.connect(self.ex
24         txt_btn=primary_button('生成路面裂缝测量分析报告'); txt_btn.clicked.connect(
25         snap_btn=QPushButton('导出当前画布标注快照'); snap_btn.clicked.connect(sel
26         self.list=QListWidget(); card.layout.addWidget(QLabel('导出记录'));
27         self.info=InfoBox('报告包含任务信息、比例尺、裂缝数量、长度宽度、等级分布、复核记录和材料估算。')
28         card.layout.addWidget(self.info); layout.addWidget(card)
29     def _tasks(self): return self.state.tasks
30     def _record(self,path): self.list.addItem(str(path)); self.info.setT
31     def check_quality(self):
32         lines=[]
33         for task in self._tasks():
34             score, details=AnnotationQualityChecker.score(task); lines.a
35         QMessageBox.information(self,'完整性检查','\n'.join(lines) if lines e
36     def export_json(self):
37         path=self.generator.export_json(self._tasks()); self._record(pat
38     def export_csv(self):
39         path=self.generator.export_csv(self._tasks()); self._record(path
40     def export_report(self):
41         path=self.generator.export_txt_report(self._tasks()); self._reco
42     def export_snapshot(self):
43         if not self.canvas: return
44         path=self.project_root/'exports'/f'canvas_snapshot.png'
45         image=self.canvas.grab(); image.save(str(path)); self._record(pa
46 from __future__ import annotations
47 from PyQt6.QtWidgets import QWidget, QVBoxLayout, QLabel, QPushButton, Q
48 from app.ui.widgets import PanelCard, primary_button, success_button, da
49 from app.core.models import ReviewLog
50 class ReviewPanel(QWidget):
51

```

```

1  def __init__(self, state, storage, project_root, canvas=None, parent
2      super().__init__(parent); self.state=state; self.storage=storage
3      layout=QVBoxLayout(self); card=PanelCard("复核纠偏舱")
4      self.comment=QLineEdit(); self.comment.setPlaceholderText("输入复核意
5      card.layout.addWidget(QLabel("复核意见")); card.layout.addWidget(sel
6      pass_btn=success_button("通过并锁定选中裂缝"); pass_btn.clicked.connect(s
7      reject_btn=danger_button("退回修改选中裂缝"); reject_btn.clicked.connect
8      unlock=QPushButton("解除锁定"); unlock.clicked.connect(self.unlock);
9      delete_line=danger_button("删除选中标注线"); delete_line.clicked.conne
10     self.list=QListWidget(); card.layout.addWidget(QLabel("复核记录"));
11     self.info=InfoBox(); card.layout.addWidget(self.info); layout.ad
12     self.state.selection_changed.connect(self.refresh); self.canvas.
13 def _log(self, result):
14     task=self.state.current_task; crack=self.state.selected_crack()
15     if not task or not crack: return None
16     log=ReviewLog(crack_id=crack.crack_id, reviewer="当前复核员", result=
17     task.review_logs.append(log); crack.review_comment=log.comment;
18 def pass_crack(self):
19     crack=self.state.selected_crack(); task=self.state.current_task
20     if not crack or not task: return
21     crack.review_status='已通过'; crack.locked=True; self._log('通过并锁定')
22     self.storage.upsert_task(task); self.canvas.update() if self.can
23 def reject_crack(self):
24     crack=self.state.selected_crack(); task=self.state.current_task
25     if not crack or not task: return
26     crack.review_status='已退回'; crack.locked=False; self._log('退回修改')
27     self.storage.upsert_task(task); self.canvas.update() if self.can
28 def unlock(self):
29     crack=self.state.selected_crack(); task=self.state.current_task
30     if not crack or not task: return
31     crack.locked=False; crack.review_status='已修正待复核'; self._log('解除锁
32     self.storage.upsert_task(task); self.canvas.update() if self.can
33 def delete_selected_line(self):
34     if not self.canvas:
35         return
36     ok, msg = self.canvas.delete_selected_any_annotation()
37     self.refresh()
38     QMessageBox.information(self, "删除标注线", msg)
39 def refresh(self):
40     self.list.clear(); task=self.state.current_task; crack=self.stat
41     if task:
42         for log in task.review_logs[-20:]: self.list.addItem(f"{log.
43         self.info.setText(f"当前裂缝: {crack.crack_id}\n 复核状态: {crack.review_st
44 from __future__ import annotations
45 from PyQt6.QtWidgets import QWidget, QVBoxLayout, QLabel, QPushButton, Q
46 from PyQt6.QtCore import Qt
47 from app.ui.widgets import PanelCard, InfoBox
48 class SnapPanel(QWidget):
49     def __init__(self, state, storage, project_root, canvas=None, parent
50         super().__init__(parent); self.state=state; self.storage=storage

```



```

1 layout=QVBoxLayout(self); card=PanelCard("磁吸辅助舱")
2 self.toggle=QPushButton("开启磁吸辅助"); self.toggle.clicked.connect(s
3 self.slider=QSlider(Qt.Orientation.Horizontal); self.slider.setR
4 card.layout.addWidget(QLabel("吸附半径")); card.layout.addWidget(sel
5 draw=QPushButton("开启磁吸后进入折线描绘"); draw.clicked.connect(self.draw_
6 self.info=InfoBox("磁吸辅助会在点击点附近寻找更暗的裂缝中心点，减少人工点位偏差。")
7 card.layout.addWidget(self.info); layout.addWidget(card)
8 def switch(self):
9     self.state.snap_assistant.set_enabled(not self.state.snap_assist
10     self.toggle.setText("关闭磁吸辅助" if self.state.snap_assistant.enable
11     self.info.setText(self.state.snap_assistant.last_message)
12     QMessageBox.information(self,"磁吸状态", self.state.snap_assistant.l
13 def radius_changed(self, value):
14     self.state.snap_assistant.set_radius(value); self.info.setText(s
15 def draw_with_snap(self):
16     self.state.snap_assistant.set_enabled(True); self.toggle.setText
17 from __future__ import annotations
18 from PyQt6.QtWidgets import QWidget, QVBoxLayout, QLabel, QPushButton, Q
19 from app.ui.widgets import PanelCard, primary_button, InfoBox
20 from app.core.statistics import RoadSectionStatistics
21 class StatisticsPanel(QWidget):
22     def __init__(self, state, storage, project_root, canvas=None, parent
23         super().__init__(parent); self.state=state; self.storage=storage
24         layout=QVBoxLayout(self); card=PanelCard("路段统计舱")
25         btn=primary_button("生成路段病害统计")
26         btn.clicked.connect(self.generate); card.layout.addWidget(btn)
27         self.risk=QProgressBar(); self.risk.setRange(0,100); card.layout
28         self.list=QListWidget(); card.layout.addWidget(QLabel('统计结果'));
29         self.info=InfoBox('统计会汇总所有任务，形成路段级裂缝总长、网裂面积、等级分布和养护优先级。')
30         card.layout.addWidget(self.info); layout.addWidget(card)
31     def generate(self):
32         stat=RoadSectionStatistics.summarize_tasks(self.state.tasks)
33         self.list.clear()
34         for k,v in stat.items():
35             if isinstance(v,dict): self.list.addItem(f'{k}: {v}')
36             elif isinstance(v,float): self.list.addItem(f'{k}: {v:.2f}')
37             else: self.list.addItem(f'{k}: {v}')
38         self.risk.setValue(stat['病害指数'])
39         advice = '优先修复' if stat['病害指数']>=70 else ('近期养护' if stat['病害指数']
40         self.info.setText(f'路段病害指数: {stat['病害指数']}\n 建议: {advice}\n 裂缝总数: {st
41         QMessageBox.information(self,'统计完成',f"病害指数: {stat['病害指数']}, 建议: {ad
42 from __future__ import annotations
43 from pathlib import Path
44 from PyQt6.QtWidgets import (
45     QWidget, QVBoxLayout, QLabel, QPushButton, QFileDialog, QLineEdit,
46     QComboBox, QListWidget, QMessageBox, QHBoxLayout
47 )
48 from PyQt6.QtCore import QTimer
49 from app.core.models import AnnotationTask
50 from app.ui.widgets import PanelCard, primary_button, InfoBox

```

```

1 class StreetViewPanel(QWidget):
2     """实时街景路面裂缝标注舱。
3     这里先做本地模拟实时街景：可导入街景图片，也可从内置街景序列逐帧切换。
4     API 接口只预留字段和测试按钮，暂不发起真实网络请求。
5     """
6     def __init__(self, state, storage, project_root, canvas=None, parent
7         super().__init__(parent)
8         self.state = state
9         self.storage = storage
10        self.project_root = Path(project_root)
11        self.canvas = canvas
12        self.street_frames = sorted((self.project_root / 'assets' / 'str
13        self.frame_index = 0
14        self.timer = QTimer(self)
15        self.timer.setInterval(1200)
16        self.timer.timeout.connect(self.next_frame)
17        layout = QVBoxLayout(self)
18        card = PanelCard('实时街景标注舱')
19        self.road_name = QLineEdit('G320 示范街景路段')
20        self.station = QLineEdit('K12+300')
21        self.location_name = QLineEdit('武汉市东西湖区示范道路')
22        self.lon = QLineEdit('114.305678')
23        self.lat = QLineEdit('30.572123')
24        self.direction = QComboBox(); self.direction.addItem('上行', '下行
25        self.road_level = QComboBox(); self.road_level.addItem('城市主干路'
26        self.note = QLineEdit('街景模拟帧，用于路面裂缝快速预标注')
27        for label, widget in [
28            ('路段名称', self.road_name),
29            ('当前桩号', self.station),
30            ('位置说明', self.location_name),
31            ('经度', self.lon),
32            ('纬度', self.lat),
33            ('道路方向', self.direction),
34            ('道路等级', self.road_level),
35            ('备注', self.note),
36        ]:
37            card.layout.addWidget(QLabel(label))
38            card.layout.addWidget(widget)
39        import_btn = primary_button('导入街景图并创建任务')
40        import_btn.clicked.connect(self.import_street_image)
41        sample_btn = QPushButton('打开真实街景样例')
42        sample_btn.clicked.connect(self.open_current_sample)
43        prev_btn = QPushButton('上一帧')
44        prev_btn.clicked.connect(self.prev_frame)
45        next_btn = QPushButton('下一帧')
46        next_btn.clicked.connect(self.next_frame)
47        card.layout.addWidget(import_btn)
48        card.layout.addWidget(sample_btn)
49        frame_row = QHBoxLayout()
50        frame_row.addWidget(prev_btn)

```

```

1     frame_row.addWidget(next_btn)
2     card.layout.addLayout(frame_row)
3     self.live_btn = QPushButton('开始模拟实时街景')
4     self.live_btn.clicked.connect(self.toggle_live)
5     card.layout.addWidget(self.live_btn)
6     api_title = QLabel('API 接入口（暂不接入，仅预留配置）')
7     card.layout.addWidget(api_title)
8     self.api_url = QLineEdit('https://your-api-endpoint.example/stre
9     self.api_key = QLineEdit('')
10    self.api_key.setPlaceholderText('API Key / Token')
11    card.layout.addWidget(QLabel('接口地址'))
12    card.layout.addWidget(self.api_url)
13    card.layout.addWidget(QLabel('API Key'))
14    card.layout.addWidget(self.api_key)
15    test_api = QPushButton('测试连接（占位，不发起网络请求）')
16    test_api.clicked.connect(self.test_api_placeholder)
17    card.layout.addWidget(test_api)
18    self.frame_list = QListWidget()
19    self.frame_list.setMaximumHeight(120)
20    self.frame_list.itemClicked.connect(self.pick_frame)
21    card.layout.addWidget(QLabel('街景帧列表'))
22    card.layout.addWidget(self.frame_list)
23    self.info = InfoBox('街景标注流程：导入/打开街景帧 → 自动标注舱识别裂缝 → 浏览模式人工修正 → 宽度
24    card.layout.addWidget(self.info)
25    layout.addWidget(card)
26    self.refresh_frames()
27    def refresh_frames(self):
28        self.frame_list.clear()
29        for p in self.street_frames:
30            self.frame_list.addItem(p.name)
31    def _create_task(self, image_path: str):
32        station = self.station.text().strip() or 'K0+000'
33        task = AnnotationTask(
34            road_name=self.road_name.text().strip() or '街景路段',
35            image_path=str(image_path),
36            pavement_type='街景路面',
37            station_start=station,
38            station_end=station,
39            lane='街景采集车道',
40            location_name=self.location_name.text().strip(),
41            longitude=self.lon.text().strip(),
42            latitude=self.lat.text().strip(),
43            road_level=self.road_level.currentText(),
44            direction=self.direction.currentText(),
45            section_note=self.note.text().strip(),
46            source_type='街景路面影像',
47            annotator='当前用户',
48            status='街景待标注',
49        )
50    self.state.add_task(task)
51

```

```

1         self.storage.upsert_task(task)
2         self.info.setText(
3             f'已创建街景标注任务: {task.road_name}\n'
4             f'位置: {task.location_name}\n'
5             f'坐标: {task.latitude}, {task.longitude}\n'
6             f'影像: {Path(task.image_path).name}\n'
7             '可进入自动标注舱执行裂缝识别。'
8         )
9     def import_street_image(self):
10         path, _ = QFileDialog.getOpenFileName(self, '导入街景路面图', str(self.
11         if path:
12             self._create_task(path)
13     def open_current_sample(self):
14         if not self.street_frames:
15             QMessageBox.warning(self, '缺少街景样例', 'assets/streetview 下没有街景
16             return
17             self._create_task(str(self.street_frames[self.frame_index]))
18     def pick_frame(self, item):
19         row = self.frame_list.currentRow()
20         if 0 <= row < len(self.street_frames):
21             self.frame_index = row
22             self.open_current_sample()
23     def next_frame(self):
24         if not self.street_frames:
25             return
26             self.frame_index = (self.frame_index + 1) % len(self.street_frame
27             self.station.setText(f'K12+{300 + self.frame_index * 50}')
28             self.open_current_sample()
29     def prev_frame(self):
30         if not self.street_frames:
31             return
32             self.frame_index = (self.frame_index - 1) % len(self.street_frame
33             self.station.setText(f'K12+{300 + self.frame_index * 50}')
34             self.open_current_sample()
35     def toggle_live(self):
36         if self.timer.isActive():
37             self.timer.stop()
38             self.state.street_view_live = False
39             self.live_btn.setText('开始模拟实时街景')
40             QMessageBox.information(self, '实时街景', '已停止模拟实时街景。')
41         else:
42             self.timer.start()
43             self.state.street_view_live = True
44             self.live_btn.setText('停止模拟实时街景')
45             QMessageBox.information(self, '实时街景', '已开始模拟连续街景帧。每一帧都会创建一个街
46     def test_api_placeholder(self):
47         QMessageBox.information(
48             self,
49             'API 接入口',
50             '当前仅预留 API 接入口, 不会发起真实网络请求。\\n'

```

```

1         f'接口地址: {self.api_url.text().strip()}\n'
2         '后续可接入街景采集车、巡检平台或第三方街景服务。'
3     )
4     from __future__ import annotations
5     from PyQt6.QtWidgets import QWidget, QVBoxLayout, QLabel, QPushButton, Q
6     from app.ui.widgets import PanelCard, primary_button, danger_button, Inf
7     from app.core.width_gauge import CrackWidthGauge
8     from app.core.auto_detect import CrackAutoDetector
9     class WidthPanel(QWidget):
10         def __init__(self, state, storage, project_root, canvas=None, parent
11             super().__init__(parent)
12             self.state = state
13             self.storage = storage
14             self.canvas = canvas
15             layout = QVBoxLayout(self)
16             card = PanelCard('宽度卡尺舱')
17             btn = primary_button('进入宽度卡尺模式')
18             btn.clicked.connect(lambda: self.state.set_mode('宽度卡尺'))
19             card.layout.addWidget(btn)
20             auto_btn = QPushButton('自动估算最窄/中间/最宽三处宽度')
21             auto_btn.clicked.connect(self.auto_estimate)
22             card.layout.addWidget(auto_btn)
23             clear = QPushButton('清空选中裂缝宽度采样')
24             clear.clicked.connect(self.clear_samples)
25             card.layout.addWidget(clear)
26             delete_line = danger_button('删除选中标注线')
27             delete_line.clicked.connect(self.delete_selected_line)
28             card.layout.addWidget(delete_line)
29             self.list = QListWidget()
30             card.layout.addWidget(QLabel('采样点列表'))
31             card.layout.addWidget(self.list)
32             self.info = InfoBox(
33                 '先在浏览模式选择人工或自动裂缝，再自动估算。系统会沿裂缝多点采样，保留最窄、中间位置、最宽三处宽度。'
34             )
35             card.layout.addWidget(self.info)
36             layout.addWidget(card)
37             self.state.selection_changed.connect(self.refresh)
38             self.canvas.canvas_changed.connect(self.refresh) if self.canvas
39             self.refresh()
40         def auto_estimate(self):
41             crack = self.state.selected_crack()
42             task = self.state.current_task
43             if not crack or not task:
44                 QMessageBox.information(self, '提示', '请先选择一条人工标注或自动识别裂缝')
45                 return
46             samples = CrackAutoDetector.estimate_widths(task.image_path, cra
47             if not samples:
48                 QMessageBox.warning(self, '估算失败', '未能自动估算宽度，请使用人工卡尺或重新标定比例尺'
49                 return
50             crack.width_samples = samples

```

```

1         if self.canvas:
2             self.canvas.refresh_all_measurements()
3             self.canvas.canvas_changed.emit()
4             self.canvas.update()
5         self.storage.upsert_task(task)
6         self.refresh()
7         vals = [s.width_mm for s in samples]
8         QMessageBox.information(
9             self,
10            '估算完成',
11            f'已基于裂缝线生成三处宽度采样: \n'
12            f'最窄: {min(vals):.2f}mm\n'
13            f'中间: {vals[len(vals)//2]:.2f}mm\n'
14            f'最宽: {max(vals):.2f}mm'
15        )
16    def clear_samples(self):
17        crack = self.state.selected_crack()
18        if not crack:
19            return
20        crack.width_samples.clear()
21        if self.canvas:
22            self.canvas.refresh_all_measurements()
23            self.canvas.canvas_changed.emit()
24        self.storage.upsert_task(self.state.current_task)
25        self.refresh()
26        QMessageBox.information(self, '已清空', '该裂缝宽度采样已清空')
27    def delete_selected_line(self):
28        if not self.canvas:
29            return
30        ok, msg = self.canvas.delete_selected_any_annotation()
31        self.refresh()
32        QMessageBox.information(self, '删除标注线', msg)
33    def refresh(self):
34        self.list.clear()
35        crack = self.state.selected_crack()
36        task = self.state.current_task
37        if not crack:
38            self.info.setText('未选择裂缝。请先在浏览模式点击裂缝线或在右侧卡片中选择。')
39            return
40        labels = ['最窄参考', '中间参考', '最宽参考']
41        for i, s in enumerate(crack.width_samples, 1):
42            label = labels[i - 1] if len(crack.width_samples) == 3 and i
43            self.list.addItem(f'{label}: {s.width_mm:.2f} mm | {s.created_a
44            length_text = '0.00'
45            if task and crack.points:
46                length_px = sum((((crack.points[i][0]-crack.points[i-1][0])*
47                length_text = f'{length_px*task.meter_per_pixel:.2f} 米' if t
48            vals = [s.width_mm for s in crack.width_samples]
49            width_detail = '暂无宽度采样'
50            if vals:

```

```

1         width_detail = f'最窄: {min(vals):.2f}mm | 中间: {vals[len(vals)//2]
2     self.info.setText(
3         f'{crack.crack_id}\n'
4         f'来源: {crack.source}\n'
5         f'长度: {length_text}\n'
6         f'{width_detail}\n'
7         f'平均宽度: {crack.avg_width_mm:.2f}mm\n'
8         f'最大宽度: {crack.max_width_mm:.2f}mm\n'
9         f'状态: {CrackWidthGauge.status_text(crack.max_width_mm)}'
10    )
11    from __future__ import annotations
12    from PyQt6.QtWidgets import QFrame, QVBoxLayout, QLabel, QPushButton, QH
13    from PyQt6.QtCore import Qt
14    def title_label(text: str) -> QLabel:
15        label = QLabel(text)
16        label.setObjectName("TitleLabel")
17        return label
18    class PanelCard(QFrame):
19        def __init__(self, title: str = "", parent=None):
20            super().__init__(parent)
21            self.setObjectName("PanelCard")
22            self.layout = QVBoxLayout(self)
23            self.layout.setContentsMargins(14, 14, 14, 14)
24            self.layout.setSpacing(10)
25            if title:
26                self.layout.addWidget(title_label(title))
27    class InfoBox(QTextEdit):
28        def __init__(self, text: str = ""):
29            super().__init__()
30            self.setReadOnly(True)
31            self.setMinimumHeight(80)
32            self.setText(text)
33    def primary_button(text: str) -> QPushButton:
34        b = QPushButton(text)
35        b.setObjectName("PrimaryButton")
36        return b
37    def success_button(text: str) -> QPushButton:
38        b = QPushButton(text)
39        b.setObjectName("SuccessButton")
40        return b
41    def danger_button(text: str) -> QPushButton:
42        b = QPushButton(text)
43        b.setObjectName("DangerButton")
44        return b
45    def make_button_row(*buttons):
46        row = QHBoxLayout()
47        for b in buttons:
48            row.addWidget(b)
49        row.addStretch()
50        return row
51

```

```

1  def group_box(title: str, layout) -> QGroupBox:
2      box = QGroupBox(title)
3      box.setLayout(layout)
4      return box
5  import sys
6  from pathlib import Path
7  PROJECT_ROOT = Path(__file__).resolve().parent
8  if str(PROJECT_ROOT) not in sys.path:
9      sys.path.insert(0, str(PROJECT_ROOT))
10 from PyQt6.QtWidgets import QApplication
11 from app.core.storage import StorageManager
12 from app.core.image_factory import ensure_sample_images
13 from app.ui.login_dialog import LoginDialog
14 from app.ui.main_window import MainWindow
15 def main():
16     ensure_sample_images(PROJECT_ROOT / 'assets' / 'samples')
17     app = QApplication(sys.argv)
18     qss_path = PROJECT_ROOT / 'assets' / 'styles' / 'light.qss'
19     if qss_path.exists():
20         app.setStyleSheet(qss_path.read_text(encoding='utf-8'))
21     storage = StorageManager(PROJECT_ROOT / 'data')
22     login = LoginDialog(storage)
23     if login.exec() != LoginDialog.DialogCode.Accepted:
24         return 0
25     window = MainWindow(storage, login.current_user, PROJECT_ROOT)
26     window.show()
27     return app.exec()
28 if __name__ == '__main__':
29     raise SystemExit(main())
30 from __future__ import annotations
31 from pathlib import Path
32 from typing import List, Tuple
33 import math
34 import numpy as np
35 from PIL import Image, ImageFilter
36 from .models import CrackObject, WidthSample
37 from .geometry import polyline_length, distance
38 from .severity import CrackSeverityEvaluator
39 Point = Tuple[float, float]
40 class CrackAutoDetector:
41     """自动裂缝标注算法。
42     本版不再使用“全局暗像素连通”作为主逻辑，而是采用：
43     1. 局部背景减法，增强“比周边更暗”的线状裂缝；
44     2. 邻域连续性过滤，去掉单个石子、纹理噪点；
45     3. 连通域几何筛选，过滤块状污渍、边缘噪声和短小纹理；
46     4. 线性支撑度验证，确认拟合折线确实落在暗线区域上；
47     5. 默认保守识别，避免一次性生成大量错误标注线。
48     """
49     PRESETS = {
50         "保守": {

```



```

1         "contrast_percentile": 96.0,
2         "min_contrast": 18,
3         "min_length_px": 130,
4         "min_area": 14,
5         "max_count": 6,
6         "support": 0.42,
7         "edge_guard": 0.035,
8         "neighbor_min": 2,
9         "line_width": 2,
10    },
11    "标准": {
12        "contrast_percentile": 94.0,
13        "min_contrast": 14,
14        "min_length_px": 95,
15        "min_area": 10,
16        "max_count": 8,
17        "support": 0.37,
18        "edge_guard": 0.025,
19        "neighbor_min": 2,
20        "line_width": 2,
21    },
22    "增强": {
23        "contrast_percentile": 91.0,
24        "min_contrast": 11,
25        "min_length_px": 70,
26        "min_area": 8,
27        "max_count": 12,
28        "support": 0.30,
29        "edge_guard": 0.018,
30        "neighbor_min": 1,
31        "line_width": 3,
32    },
33 }
34 @staticmethod
35 def detect_from_image(
36     image_path: str | Path,
37     meter_per_pixel: float | None = None,
38     max_cracks: int | None = None,
39     min_length_px: float | None = None,
40     sensitivity: str = "保守",
41 ) -> List[CrackObject]:
42     arr = CrackAutoDetector._load_gray(image_path)
43     if arr is None:
44         return []
45     cfg = dict(CrackAutoDetector.PRESETS.get(sensitivity, CrackAutoD
46     if max_cracks is not None:
47         cfg["max_count"] = max_cracks
48     if min_length_px is not None:
49         cfg["min_length_px"] = min_length_px
50     small, scale = CrackAutoDetector._downsample(arr)

```